

T-ESP-800 · EPITECH EIP

Pawrise Care

Dossier de conception & documentation technique

0 Sommaire

1	Résumé exécutif	4
2	Vision & problématique	5
2.1	Le problème	5
2.2	La solution Pawrise Care	5
2.3	Proposition de valeur	5
2.4	Personas	6
3	Marché & analyse stratégique	7
3.1	Proposition de valeur	7
3.2	Marché (TAM / SAM / SOM)	7
3.3	SWOT	8
3.4	Matrice de confrontation	8
3.5	PESTEL	9
4	Modèle économique & budget	10
4.1	Modèle économique	10
4.2	Unit economics (3 scénarios)	10
4.3	Budget prévisionnel · collier (BOM)	11
4.4	Coût annuel de l'infrastructure	11
5	Exigences produit (PRD)	14
5.1	Exigences fonctionnelles (FR1 à FR43)	14
5.2	Exigences non fonctionnelles (NFR1 à NFR10)	15
6	WBS & fonctions par epic	16
6.1	User stories & critères d'acceptation	16
7	Organisation & compétences	18
7.1	OBS · organigramme des responsabilités	18
7.2	Couverture nominative	18
7.3	Matrice de compétences (extrait)	19
7.4	Justification du recrutement	20
7.5	Compétences non couvertes par l'équipe	21
8	Matrice RACI	22
8.1	Règle appliquée	22
8.2	Version standard (A distribué)	22
8.3	Version format suiveur (1 R / 1 A / 1 C)	22
8.4	Contrôle de cohérence	23
9	Planning & méthodologie	24
9.1	Vue d'ensemble	24
9.2	Diagramme de Gantt	24
9.3	Jalons	26
9.4	Séquencement & dépendances	26
9.5	Méthodologie	26
9.6	Outil de pilotage (Jira)	27
10	Plan qualité	28
10.1	Definition of Done	28
10.2	Stratégie de tests par couche	28
10.3	Couverture de code : objectif & seuil	28
10.4	Conventions de code	29

10.5	Workflow Git	29
10.6	Intégration continue (CI)	29
10.7	Sécurité des dépendances & supply chain	30
10.8	Déploiement (CD)	30
10.9	Onboarding développeur	30
11	Architecture technique	31
11.1	Vue d'ensemble	31
11.2	Choix techniques justifiés	33
12	Cloud & infrastructure	35
12.1	Principes directeurs	35
12.2	Dimensionnement (issu de l'architecture)	35
12.3	Comparaison des options d'hébergement	35
12.4	Souveraineté : pourquoi l'UE	36
12.5	Stratégie en deux temps	36
12.6	La stack d'infrastructure	36
12.7	Vue plateforme : topologie de déploiement	37
12.8	Maîtrise des coûts en développement	40
13	IoT · le collier connecté	41
13.1	Les défis à résoudre	41
13.2	Choix du microcontrôleur : nRF9160 (SiP)	41
13.3	Composants embarqués	41
13.4	Capteurs : choix justifiés	42
13.5	Réseau & sécurité	42
13.6	Logique embarquée : machine à états	42
13.7	Du prototype au produit	42
14	IA · le Care Engine	43
14.1	Pipeline LangGraph (6 nœuds)	43
14.2	Garde-fous (défense en profondeur, 3 couches)	43
14.3	Périmètre	43
14.4	Coût des modèles (cascade « un modèle par nœud »)	43
14.5	Décisions d'architecture (ADR)	44
14.6	Objectifs de succès (KPIs MVP)	44
15	Gestion des risques	45
15.1	AMDEC produit	45
15.2	Risk Map projet	45
16	Annexes	47
16.1	Backlog complet · 66 user stories	47
16.2	Glossaire	48
16.3	Références des décisions d'architecture (ADR)	49
16.4	Sources & outils	49
16.5	Prototype en ligne	49

1 Résumé exécutif

Pawrise Care est un collier connecté qui suit en continu la santé et le bien-être d'un animal, l'explique au propriétaire via un assistant IA, et transmet au vétérinaire un dossier structuré et normalisé. Le positionnement est non négociable : **orienter, jamais diagnostiquer**.

Là où les acteurs établis (Tractive, Weenect, Invoxia) se concentrent sur la localisation, Pawrise Care ouvre la passerelle propriétaire vers vétérinaire que personne ne couvre : c'est une stratégie océan bleu sur un marché adressable de 26,5 M de chiens et chats en France.

11

epics

66

user stories (52 MVP)

10

membres · 5 pôles

20 mois

de roadmap

Ce dossier réunit l'ensemble des livrables de conception : vision et marché, modèle économique et budget, découpage (WBS) et organisation (OBS, RACI), planning et méthodologie, plan qualité, gestion des risques, et l'architecture technique complète (système, cloud, IoT, IA). Une maquette interactive du produit (architecture jouable, diagrammes et contenus) est par ailleurs consultable sur notre site pawrise-care.com.

CE QUI DISTINGUE PAWRISE CARE

Une IA d'orientation conçue conforme dès l'origine (Code rural, AI Act, RGPD), un réseau vétérinaire réel, un modèle économique dont le coût vétérinaire est plafonné (une part fixe de l'abonnement finance le réseau : la dépense vétérinaire ne peut donc pas dépasser ce budget), et une architecture souveraine (UE) maîtrisée en coût.

2 Vision & problématique

2.1 Le problème

Les signaux de santé d'un animal de compagnie passent le plus souvent inaperçus jusqu'à ce que la situation devienne grave. Entre deux visites vétérinaires, le propriétaire n'a aucune visibilité continue sur l'état réel de son animal, et le vétérinaire reçoit un historique imprécis, reconstitué de mémoire. Les signaux faibles (baisse d'activité, troubles du sommeil, variations de constantes) sont donc détectés trop tard, souvent au stade critique.

Les colliers connectés ont beaucoup progressé : au-delà du GPS, plusieurs acteurs (Invoxia, Tractive) embarquent désormais des capteurs de santé (activité, fréquence cardiaque et respiratoire) et des alertes. Mais tous s'arrêtent à la donnée brute : ils mesurent et affichent des courbes, sans les expliquer ni accompagner le propriétaire. La valeur manquante n'est pas un capteur de plus, c'est l'interprétation.

2.2 La solution Pawrise Care

Pawrise Care ne cherche pas à gagner la course aux capteurs. Sa valeur unique est l'**explicabilité** : un assistant IA, le Care Engine, traduit les données du collier en langage clair pour le propriétaire, contextualise les alertes et répond à ses questions. Et quand une situation le justifie, un réseau de vétérinaires partenaires prend le relais avec un dossier déjà préparé. La chaîne de valeur est claire : le collier mesure, l'IA explique et oriente, le vétérinaire décide.

2.3 Proposition de valeur

Notre différenciation n'est pas le nombre de capteurs mais l'interprétation accompagnée, là où les concurrents s'arrêtent à la donnée brute. Elle repose sur deux piliers :

- **Explicabilité par l'IA** : l'assistant conversationnel explique l'état de l'animal en clair et contextualise les alertes. Une valeur difficile à copier car elle s'appuie sur un corpus vétérinaire validé et des garde-fous.
- **Relais vétérinaire** : dès qu'un avis médical devient nécessaire, un vétérinaire partenaire prend le relais avec un dossier normalisé prêt à l'emploi. Ce réseau est rémunéré par une part fixe des abonnements (pool), ce qui plafonne le coût vétérinaire par construction (voir modèle économique).

C'est un positionnement **océan bleu** : ne pas livrer la guerre des capteurs, mais ouvrir la passerelle propriétaire vers vétérinaire que les acteurs établis ne couvrent pas.

RÈGLE D'OR, NON NÉGOCIABLE

L'assistant IA n'établit **jamais** de diagnostic médical (Code rural, article L243-1) : il explique, oriente, et invite à consulter. C'est précisément le réseau vétérinaire qui prend le relais dès qu'un acte médical est requis. Toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

2.4 Personas

Deux personas primaires, les deux faces de la plateforme, volontairement approfondis.

Propriétaire d'animal

25 à 55 ans, attaché à son animal, sans formation vétérinaire, accès vétérinaire parfois difficile (zone rurale, planning saturé).

Objectifs : comprendre l'état de son animal au quotidien, agir au bon moment, éviter l'urgence.

Frustrations : données brutes illisibles, incertitude « est-ce grave ? », délais et coût des consultations.

Vétérinaire partenaire

En cabinet ou en téléconsultation, prudent face aux outils IA généralistes, intéressé par la préparation en amont des consultations.

Objectifs : préparer ses consultations, gagner du temps, fidéliser sa clientèle.

Frustrations : historiques imprécis, données brutes non exploitables, sollicitations non filtrées.

Segments prioritaires (early adopters) : propriétaires en zone d'accès vétérinaire limité, foyers multi-animaux, propriétaires de races à risques de santé connus. Un acteur secondaire « système / administration » (provisioning des colliers, RGPD, exploitation) est traité dans les parties Organisation et Architecture.

3 Marché & analyse stratégique

3.1 Proposition de valeur

Les concurrents (Tractive, Weenect, Invoxia) ne se limitent plus au GPS : ils embarquent aussi des capteurs de santé. Mais ils s'arrêtent à la donnée brute. Pawrise Care **interprète** : GPS et capteurs de santé, plus une lecture accompagnée par l'IA et le vétérinaire pour orienter le propriétaire vers le bon soin, au bon moment. C'est un positionnement **océan bleu** : ne pas livrer la guerre des capteurs, mais ouvrir la passerelle propriétaire vers vétérinaire (dossier normalisé, suivi longitudinal) que les acteurs établis ne couvrent pas.

Trois piliers : **Prévention** (détecter les signaux faibles avant qu'ils n'empirent), **Orientation non-diagnostique** (aide à l'observation, jamais un acte médical), **Réseau vétérinaire** (escalade vers un professionnel avec un dossier prêt à l'emploi, rémunéré par une part fixe des abonnements).

3.2 Marché (TAM / SAM / SOM)

On dimensionne le marché en entonnoir : le **TAM** (Total Addressable Market) est le marché total si l'on captait tout le monde ; le **SAM** (Serviceable Addressable Market) la part que notre produit peut réellement adresser ; le **SOM** (Serviceable Obtainable Market) la part réalismement captable à court/moyen terme.

≈ 73 M

TAM · tous animaux de compagnie en France

≈ 26,5 M

SAM · chiens (9,9 M) + chats (16,6 M)

1 foyer / 2

possède au moins un animal

3-5 ans

SOM · dizaines de milliers d'abonnés

TAM ≈ 73 M : l'ensemble des animaux de compagnie en France, poissons, oiseaux et NAC compris. **SAM ≈ 26,5 M** : les chiens et chats (9,9 M + 16,6 M), seuls réellement adressables par un collier connecté. **SOM** : quelques dizaines de milliers d'abonnés visés sur 3 à 5 ans, compte tenu de la notoriété à bâtir, du budget et de la concurrence. Le repère « 1 foyer sur 2 » (part des foyers français possédant un animal) illustre la profondeur du marché. Le marché pet tech est en forte croissance, porté par l'humanisation des animaux ; les concurrents mesurent désormais la santé mais restent sur la donnée brute, l'interprétation accompagnée (IA + vétérinaire) reste notre différenciation.

Sources : [FACCO / Kantar, chiffres clés 2024-2025](#) ; rapports de marché pet tech ([Grand View Research](#), [pet wearables](#)).

3.3 SWOT

Forces

Santé expliquée et accompagnée : la donnée brute des concurrents traduite en clair par l'IA, relayée par un vétérinaire.

Expertise IA + garde-fous non-diagnostiques : RAG/LLM maîtrisés en interne, conformité intégrée dès le départ.

Vétérinaire partenaire réel : crédibilité scientifique et accès au métier.

Données longitudinales propriétaires : un actif santé par animal qui grossit avec le temps.

Faiblesses

Hardware complexe et coûteux : conception d'un collier fiable, lourde en temps et budget.

Démarrage à froid des données : fiabilité de l'IA dépendante de données à accumuler.

Prototype non finalisé : encore sur capteurs simulés, risque technique avant produit réel.

Rentabilité à prouver : modèle d'abonnement dont la viabilité reste à valider.

Opportunités

Marché français massif et porteur : 26,5 M de chiens et chats, segment connecté en croissance.

Humanisation des animaux : disposition réelle à payer pour la santé et la prévention.

Assurance animale en croissance : nos données aident la prévention et la tarification : canal + revenu B2B.

Interopérabilité via le PDF normalisé : gain de temps vétérinaire, adoption facilitée.

Menaces

Concurrents déjà sur la santé : Invoxia, Tractive mesurent déjà FC/FR : angle santé non vierge.

Acteur capitalisé qui copie la couche vétérinaire : un géant pourrait aller plus vite.

Cadre réglementaire strict et mouvant : acte médical réservé au vétérinaire, IA encadrée (AI Act).

Adoption lente des vétérinaires : profession prudente et surchargée.

Sources : marché FACCO / Kantar ; santé embarquée des concurrents [Invoxia](#), [Tractive](#) ; cadre réglementaire [AI Act](#) (UE 2024/1689), [Code rural](#), art. L243-1.

3.4 Matrice de confrontation

Ce qu'on fait du SWOT : croiser forces et faiblesses avec opportunités et menaces pour en tirer des axes d'action.

Axe	Stratégie
Offensif · Forces × Opportunités	Viser les assureurs animaliers : le suivi continu et le dossier vétérinaire réduisent leurs sinistres, nos données nourrissent leur tarification. On gagne un canal de distribution et un revenu B2B.
Défensif · Forces × Menaces	Ne pas livrer la guerre des capteurs (déjà gagnée par Tractive/Invoxia) : se protéger sur l'orientation accompagnée et le réseau vétérinaire, non copiables sans partenariats longs à nouer.
Rattrapage · Faiblesses × Opportunités	Transformer le démarrage à froid en boucle de données : les premiers cabinets et assureurs partenaires apportent cas réels et co-financement, fiabilisant l'IA et le matériel plus vite.
Vigilance · Faiblesses × Menaces	Sécuriser les points faibles avant de grandir : LLM derrière une couche d'abstraction, garde-fous documentés pour encaisser un durcissement réglementaire, rentabilité prouvée sur un périmètre réduit.

3.5 PESTEL

L'analyse macro-environnementale, fil conducteur : orienter, jamais diagnostiquer.

Axe	Synthèse	À retenir
Politique	France 2030, BPI et UE financent l'IA et l'e-santé ; le bien-être animal est une priorité publique (loi du 30/11/2021) ; souveraineté des données.	Terrain favorable si hébergement souverain UE.
Économique	Marché animalier large et résilient ; pet tech en croissance. Vigilance : abonnement récurrent (~10 €/mois) dans un contexte de lassitude.	Maîtrise du churn et du CAC décisive.
Socioculturel	L'animal, membre de la famille ; demande de prévention proactive ; usages numériques entrés dans les mœurs.	Demande qui colle à notre proposition.
Technologique	Convergence capteurs / IA / cloud / télémédecine. Réserve : mesurer des constantes fiables sur un animal qui bouge.	Fiabilité de la mesure = risque central.
Environnemental	Éco-conception imposée (Règlement Batteries UE 2023/1542, DEEE, indice de réparabilité) ; sobriété numérique (Green IT).	Durabilité matériel + sobriété numérique.
Légal	RGPD, Code rural (diagnostic réservé aux vétérinaires), AI Act, sécurité produit (GPSR UE 2023/988).	Tout repose sur la frontière orienter / diagnostiquer.

Sources officielles : France 2030, loi n° 2021-1539 (bien-être animal), FACCO / Kantar ; RGPD, Code rural L243-1, AI Act, GPSR (UE 2023/988), Règlement Batteries (UE 2023/1542), indice de réparabilité.

4 Modèle économique & budget

4.1 Modèle économique

Le hardware est vendu proche de son coût ; la rentabilité vient de l'**abonnement récurrent**.

Le modèle intègre un **pool de rémunération façon Spotify** : une **part fixe** de chaque abonnement alimente une cagnotte qui rémunère les vétérinaires, répartie au prorata de leur activité. Conséquence : le coût vétérinaire total ne peut jamais dépasser ce pourcentage du chiffre d'affaires, quel que soit le nombre de sollicitations. Il est donc plafonné « par construction » (mathématiquement), ce qui protège la marge : la dépense vétérinaire grandit avec les revenus, jamais plus vite. Des garde-fous (filtrage IA avant escalade, usage équitable) évitent l'abus.

D'où vient le prix ? On part pour l'instant sur **9,90 €/mois**, à faire évoluer au besoin. On se positionne volontairement dans le haut du marché (les concurrents facturent 5 à 13 €/mois) : notre valeur ajoutée (interprétation par l'IA et relais vétérinaire) justifie cet alignement par le haut.

Les termes, en clair :

- **Marge contributive** : ce qu'il reste de l'abonnement une fois payés les coûts directs d'un client (cloud, API d'IA, pool vétérinaire). Exemple : sur 9,90 €, s'il reste 6,80 €, la marge est de 69 %.
- **Churn mensuel** : le pourcentage de clients qui résilient chaque mois. S'il est de 3 %/mois, un client reste en moyenne $1 / 0,03 \approx 33$ mois : c'est la **durée de vie client**.
- **LTV (valeur vie client)** : ce qu'un client rapporte au total sur toute sa durée de vie, soit abonnement $\times$ nombre de mois $\times$ marge. Exemple : $9,90 \text{ €} \times 30 \text{ mois} \times 69 \% \approx 205 \text{ €}$.
- **CAC (coût d'acquisition)** : ce qu'il faut dépenser (publicité, commercial) pour gagner un client.

4.2 Unit economics (3 scénarios)

Les **unit economics** répondent à une seule question : un client, à lui seul, rapporte-t-il plus qu'il ne coûte ? On encadre l'incertitude avec trois scénarios (optimiste, réaliste, pessimiste).

Comment les scénarios sont bâtis. On fixe trois **leviers d'entrée** à partir de repères du marché de l'abonnement grand public : le **churn** (taux de résiliation mensuel, 2 % / 3 % / 8 % par mois, d'où une durée de vie de 48 / 30 / 12 mois), le **CAC** (35 / 50 / 90 €, repères d'acquisition digitale) et la **marge** (78 / 69 / 35 %, selon ce que consomment cloud, IA et pool vétérinaire). Les deux dernières lignes, **LTV : CAC** et **Payback**, en sont ensuite calculées. Optimiste = meilleurs niveaux plausibles ; base = prudent et réaliste ; pessimiste = combinaison défavorable servant de garde-fou. C'est un modèle prévisionnel, à affiner avec les chiffres réels une fois en marché.

Métrique	Optimiste	Base	Pessimiste
Marge contributive (abo)	78 %	69 %	35 %
Durée de vie (churn)	≈ 48 mois	≈ 30 mois	≈ 12 mois
CAC	$\approx 35 \text{ €}$	$\approx 50 \text{ €}$	$\approx 90 \text{ €}$
LTV : CAC	$\approx 10\times$	$\approx 4\times$	$\approx 0,5\times$
Payback	≈ 5 mois	≈ 7 mois	non atteint *

Comment lire. LTV : CAC compare ce qu'un client rapporte (LTV) à ce qu'il coûte à acquérir (CAC) : au-dessus de $3\times$ le modèle est sain, à $1\times$ on est à l'équilibre, en dessous on perd de l'argent. **Payback** : le nombre de mois pour rembourser le coût d'acquisition grâce à la marge mensuelle. Lecture du scénario de base : un client rapporte environ $4\times$ sa mise, remboursé en 7 mois. Le scénario pessimiste (churn et CAC élevés) ne rembourse jamais : c'est celui à éviter, en réduisant le churn et le coût d'acquisition.

4.3 Budget prévisionnel · collier (BOM)

Nomenclature du produit final (PCB custom, assemblage JLCPCB), en prototype unitaire et en série de 1000. Choix clé : le nRF9160 (SiP MCU + modem) qui supprime les AT commands. Composants détaillés en partie IoT.

Composant	Prototype ×1	Série ×1000
nRF9160-SICA · MCU + modem LTE-M/NB-IoT (SiP)	12 €	10 €
u-blox ZOE-M8Q · GNSS multi-constellation	14 €	10 €
LSM6DSOX · centrale inertielle (IMU)	2,85 €	1,85 €
TMP117 · température ±0,1 °C	2,50 €	1,45 €
Fréquence cardiaque · 1 option (conditionnel R&D)	0,80 à 6,50 €	0,45 à 4,20 €
W25Q128 · mémoire flash 16 Mo	1,20 €	0,80 €
BQ25895 + TLV62585 · PMU (charge, fuel gauge, régulateur)	5,35 €	3,55 €
Antennes Taoglas FPC (LTE-M + GNSS)	4,70 €	2,70 €
Connecteurs (USB-C, JST)	0,85 €	0,57 €
Composants passifs (R/C/L, quartz, LED, bouton)	6,10 €	2,00 €
PCB 4 couches + assemblage PCBA (JLCPCB)	38 €	6,50 €
Batterie Li-Po 2000 mAh	12 €	8 €
SIM IoT 1NCE (500 Mo / 10 ans)	10 €	10 €
Boîtier IP67 (injection) + étanchéité + mécanique	25,60 €	5,30 €

Total : ≈ 131 € (prototype) · ≈ 66 €/u (série)

4.4 Coût annuel de l'infrastructure

Coût d'exploitation sur **un an**, en production une fois le produit lancé, développement inclus. Hébergement souverain (UE), auto-géré via Terraform et Kubernetes. Le dimensionnement retenu (≈ 16 vCPU / 32 Go) découle directement de l'architecture (services Rust, bases de données, bus d'événements, broker MQTT, moteur IA, observabilité), justifié en partie Cloud.

Hypothèses de calcul : lancement à faible trafic et usage IA modéré ; prix serveurs = tarifs publics Hetzner de juin 2026 (ligne ARM CAX) ; parité euro/dollar retenue pour l'IA ; certificats TLS gratuits (Let's Encrypt).

Détail des postes

Poste	Détail	Coût annuel
Cluster Kubernetes (auto-géré)	1× CAX21 control-plane + 2× CAX31 workers (ARM Ampere, 8 vCPU / 16 Go)	≈ 630 €/an
Sauvegardes & volumes	volumes bloc ~150 Go (0,057 €/Go) + backups automatiques (+20 %)	≈ 230 €/an
Load balancer + IP publique	LB11 (répartition de charge, TLS)	≈ 72 €/an
Sauvegardes hors-site	Storage Box (rétention longue durée)	≈ 60 €/an
Modèle de langage (IA)	Azure OpenAI + Cohere · ~0,03 à 0,05 €/conversation (variable)	≈ 600 €/an
Nom de domaine	.com (TLS Let's Encrypt gratuit)	≈ 12 €/an
Développement & tests	dev en local (gratuit) + CI ; tests éphémères et IA à la consommation	≈ 150 €/an

Total : ≈ 1 750 €/an

Comment chaque poste est chiffré :

- **Compute (cluster)** : 1× CAX21 (10,49 €) + 2× CAX31 (2 × 20,99 €) = 52,47 €/mois, soit ≈ 630 €/an. Taille calée sur le besoin de 16 vCPU / 32 Go.
- **Sauvegardes & volumes** : 150 Go de volumes bloc (0,057 €/Go) + sauvegardes automatiques (+20 % du prix des instances) ≈ 19 €/mois, soit ≈ 230 €/an.
- **Load balancer + IP publique** : LB11 à 5,99 €/mois, soit ≈ 72 €/an.
- **Sauvegardes hors-site** : Storage Box (rétention longue durée) 5 €/mois, soit ≈ 60 €/an.
- **Modèle de langage (IA)** : 0,03 à 0,05 € par conversation ; poste **variable**, ≈ 600 €/an à faible volume, qui croît avec les abonnements qui le financent.
- **Nom de domaine** : ≈ 12 €/an (TLS gratuit).

Sources des prix : [Hetzner Cloud](#) + ajustement du 15/06/2026 (CAX21 10,49 €, CAX31 20,99 €/mois ; volumes 0,057 €/Go) ; IA : [Azure OpenAI](#), [Cohere](#).

Développement & tests (à la consommation)

L'essentiel du développement ne coûte rien, et les tests sont payés à l'usage :

- **Développement en local** : 0 €. Chaque développeur lance la stack sur sa machine (Docker Compose / kind).
- **Intégration continue** : GitHub Actions (offre incluse pour le projet).
- **Environnement de test / staging éphémère** : un petit cluster (31 €/mois en continu) n'est actif que 40 h/semaine, détruit le soir et le week-end via terraform destroy (24 % du temps), soit ≈ 7,5 €/mois → ≈ 90 €/an.
- **Consommation IA de test** : 5 €/mois à l'usage, soit ≈ 60 €/an.

Total développement ≈ **150 €/an**. Un serveur arrêté restant facturé chez Hetzner, l'économie vient de la destruction/recréation par IaC, pas de l'extinction (mécanisme détaillé en partie Cloud).

Comparaison (coût annuel, capacité équivalente)

Option d'hébergement	Coût annuel*
Bare metal (serveur dédié)	~1 000 à 1 500 €
Hetzner Cloud auto-géré (retenu)	≈ 1 750 € (dont ~150 € de dev)
Cloud managé EU (Scaleway / OVH)	~3 000 à 3 700 €
Hyperscaler (AWS / Azure / GCP)	~6 000 à 10 000 €

* Capacité équivalente (16 vCPU / 32 Go + bases de données, load balancer, sauvegardes et IA), faible trafic. Critères complets (souveraineté, exploitation, lock-in) en partie Cloud.

Pourquoi pas le bare metal : il reste acceptable pour des tests ponctuels, mais il n'est pas maintenable en production. Il ajoute toute une couche de complexité opérationnelle dont on ne veut absolument pas : gestion physique des serveurs, remplacement des disques et du matériel défaillant, redondance à provisionner et câbler en double, mises à jour firmware, supervision matérielle, et aucune élasticité pour absorber un pic ou créer un environnement à la demande. Le gain sur le compute brut ne compense pas cette charge d'exploitation permanente. On le réserve donc, au mieux, à des tests, jamais à la production.

Pourquoi pas l'hyperscaler aujourd'hui : notre charge de lancement ne le justifie tout simplement pas. Il coûte 4 à 6× plus cher, avec un fort lock-in et une souveraineté seulement partielle (le Cloud Act américain peut s'appliquer même sur une région européenne). Y aller avant d'en avoir le besoin serait du sur-engineering. On le garde en réserve pour un éventuel besoin multi-région ou un SLA entreprise.

Retenu : Hetzner Cloud auto-géré, le meilleur compromis coût / souveraineté / élasticité pour notre charge actuelle ; le cloud managé européen prend le relais au moment du passage à l'échelle.

Prudence des chiffres (scénarios majorants)

Tous les montants présentés sont volontairement des majorants (hypothèses pessimistes), afin de ne jamais sous-estimer le budget :

- **Développement** : le calcul suppose un environnement de test actif environ 40 h par semaine, chaque semaine. En réalité, nous ne développerons pas en continu, et un environnement de staging ne sera monté que lors des vraies phases de test, bien plus rarement. Le coût réel de développement sera donc nettement inférieur aux ≈ 150 €/an affichés.
- **Intelligence artificielle** : le coût de 0,03 à 0,05 € par conversation est une fourchette haute, délibérément peu optimiste. Les optimisations prévues (mise en cache, cascade de modèles, modèles moins chers sur les nœuds simples) le feront baisser. Surtout, le poste IA est encadré par des **alertes de consommation** et des **plafonds journaliers** : au-delà d'un seuil, le service se dégrade gracieusement ou bascule sur une réponse de repli, ce qui rend tout dérapage budgétaire impossible.

Autrement dit, le total de ≈ 1 750 €/an est un plafond prudent, pas une projection optimiste : le coût réel devrait être sensiblement inférieur.

Ce qui varie avec l'échelle

Seul le poste IA est réellement variable : il croît avec le nombre de conversations, donc avec les abonnements qui le financent, ce qui garde ce coût couvert par la marge. Le compute, lui, augmente par paliers (ajout de workers) et bascule vers un cloud managé européen au moment du passage à l'échelle (voir partie Cloud). L'infrastructure reste ainsi maîtrisée et proportionnée à l'usage.

5 Exigences produit (PRD)

Ce chapitre constitue le cœur du **Product Requirements Document (PRD)** : il définit **ce que le produit doit faire** et **les qualités qu'il doit garantir**. La vision, le problème et les personas figurent en partie Vision ; les exigences ci-dessous constituent le **référentiel** du découpage du travail (WBS, partie suivante).

On distingue les **exigences fonctionnelles (FR)**, ce que le système doit faire, et les **exigences non fonctionnelles (NFR)**, les qualités et contraintes. Chaque exigence porte un **code unique**, un **énoncé testable**, une **méthode de vérification**, une **priorité** (MVP ou post-MVP) et l'**epic** qui la couvre ; les deux dernières colonnes assurent la traçabilité (aucune exigence orpheline) et le contrôle qualité. Les bandeaux de domaine reprennent les couleurs du WBS.

5.1 Exigences fonctionnelles (FR1 à FR43)

Code	Exigence	Vérification	Prio.	Epic
Collier & collecte				
FR1	Localisation GPS de l'animal	Test terrain (écart de position)	MVP	E2
FR2	Mesure de l'activité (accéléromètre)	Test capteur vs référence	MVP	E2
FR3	Mesure de la température corporelle	Étalonnage vs thermomètre	MVP	E2
FR4	Mesure du rythme cardiaque (option)	Comparaison capteur médical	MVP	E2
FR5	Collecte périodique et temps réel	Test de fréquence d'échantillonnage	MVP	E2
FR6	Tampon local en cas de perte réseau	Test hors-ligne / reconnexion	MVP	E2
FR7	Transmission sécurisée et gestion batterie	Test de charge + autonomie	MVP	E2
Backend & données				
FR8	Ingestion et normalisation des données	Tests d'intégration API	MVP	E2
FR9	Stockage en séries temporelles	Tests requêtes / rétention	MVP	E2
FR10	Calcul d'agrégats et d'indicateurs	Tests unitaires de calcul	MVP	E2
FR11	API de données pour l'application	Tests d'intégration API	MVP	E3
FR12	Gestion des comptes, animaux et accès	Tests d'autorisation (abus)	MVP	E1
Moteur IA				
FR13	Prétraitement et fenêtrage des données	Tests unitaires du pipeline	MVP	E3
FR14	Modèle de comportement normal	Jeu d'évaluation par animal	MVP	E5
FR15	Détection d'anomalies (seuils adaptatifs)	Jeu d'éval + taux faux positifs	MVP	E5
FR16	Assistant conversationnel (chat RAG)	Golden set Q/R, recall	MVP	E6
FR17	Escalade non-diagnostique	Tests garde-fous (0 diagnostic)	MVP	E6
FR18	Rapport structuré (PDF / JSON)	Validation du format	MVP	E7
App propriétaire				
FR19	Écran bien-être / état du jour	Tests UI + revue UX	MVP	E3
FR20	Localisation temps réel et historique	Test E2E carte	MVP	E4
FR21	Zones de sécurité (geofencing) et alertes	Test franchissement de zone	MVP	E4
FR22	Notifications et rappels de soins	Test déclenchement d'alertes	MVP	E5
FR23	Accès au chat IA et mise en relation	Test E2E du parcours	MVP	E6
Vet Portal				
FR24	Vue patient (dossier regroupé)	Tests UI + revue vétérinaire	MVP	E7
FR25	Historique médical chronologique	Test d'affichage	MVP	E7
FR26	Résumé contextualisé et données brutes	Revue véto de pertinence	MVP	E7
FR27	Alertes, rapport et ajout de notes	Test E2E portail	MVP	E7
FR28	Export et suivi longitudinal	Validation de l'export	MVP	E7
Cycle de vie & business				
FR29	Création de compte et onboarding	Test E2E inscription	MVP	E1
FR30	Appairage du collier (BLE / QR code)	Test d'appairage BLE	MVP	E1

Code	Exigence	Vérification	Prio.	Epic
FR31	Abonnement et facturation	Test paiement (sandbox Stripe)	post	E8
FR32	Support client et FAQ	Test parcours support	post	E8
FR33	Profil animal, multi-animaux, remplacement	Tests fonctionnels	MVP	E1
FR34	Onboarding des vétérinaires partenaires	Test parcours véto	MVP	E7
Téléconsultation & triage				
FR35	PDF normalisé pour le vétérinaire	Validation du format	post	E10
FR36	Handoff structuré IA vers vétérinaire	Test E2E IA vers véto	post	E10
FR37	Orientation non-diagnostique (triage)	Tests garde-fous	post	E10
FR38	File de garde vétérinaire	Test de routage	post	E10
FR39	Prise de rendez-vous	Test de booking	post	E10
FR40	Pool de rémunération plafonné	Test de plafond	post	E10
FR41	Garde-fous et contrôle qualité	Tests adversariaux	post	E10
FR42	Routage vers le vétérinaire traitant	Test de routage	post	E10
FR43	Traçabilité et audit des téléconsultations	Vérification des logs	post	E10

5.2 Exigences non fonctionnelles (NFR1 à NFR10)

Qualités et contraintes transverses du système, regroupées par catégorie (toutes en périmètre MVP).

Code	Exigence	Vérification	Epic
Qualité de service			
NFR1	Fiabilité : disponibilité et tolérance aux pannes	Tests de charge / chaos, SLA	E11
NFR6	Performance : temps de réponse maîtrisés	Tests de performance (p95)	E11
NFR7	Scalabilité / Ops : montée en charge et exploitabilité	Tests de montée en charge	E11
Sécurité, données & éthique			
NFR3	Sécurité : chiffrement, authentification, mTLS	Audit / pentest, revue de code	E9
NFR4	Confidentialité / RGPD : consentement, minimisation, droits	Revue conformité, DPIA	E9
NFR5	Éthique IA : non-diagnostic, garde-fous, transparence	Golden set anti-diagnostic	E9
Matériel & énergie			
NFR2	Autonomie / énergie : autonomie du collier et sobriété	Mesure d'autonomie terrain	E2
NFR8	Robustesse physique : IP67, chocs et températures	Tests IP67, chocs, thermiques	E2
Qualité produit & code			
NFR9	Utilisabilité : simplicité d'usage et accessibilité	Tests utilisateurs	E3
NFR10	Maintenabilité : qualité de code, tests, documentation	Revue de code, couverture tests	E11

6 WBS & fonctions par epic

Ce **Work Breakdown Structure (WBS)** décompose le **périmètre total** du projet de façon **hiérarchique et orientée livrables**. Trois niveaux : le projet Pawrise Care (niveau 0), ses **11 epics** livrables (niveau 1, codes 1 à 11), et les **fonctions** attendues de chaque epic (niveau 2, codes 1.1, 1.2...). Les 66 user stories et leurs critères d'acceptation (Given/When/Then) affinent le niveau 2. Les codes FR/NFR affichés renvoient au référentiel défini en partie **Exigences produit (PRD)** ; la **traçabilité** epic vers exigences est portée par chaque carte ci-après (ligne « Exigences »).

Le diagramme unique ci-après (page suivante, en paysage) présente cette décomposition et, pour chaque livrable, décrit les **fonctions attendues** ainsi que les **exigences** qu'elles couvrent, afin d'en faciliter l'implémentation. Les codes FR/NFR renvoient au référentiel de la partie **Exigences produit (PRD)**. Périmètre MVP : 9 epics (52 US) ; post-MVP : epics 8 et 10.

6.1 User stories & critères d'acceptation

Chaque epic (niveau 1) se décline en **user stories** (66 au total, liste complète en annexe) ; chacune porte des **critères d'acceptation** testables au format Given/When/Then, qui rendent la fonction vérifiable et prête à implémenter. Exemple :

STORY 1.1 · CRÉATION DE COMPTE SÉCURISÉE

En tant que propriétaire d'animal, **je veux** créer un compte avec email et mot de passe, **afin d'**accéder à l'application en toute sécurité.

Given un visiteur sur l'écran d'inscription

When il saisit un email valide et un mot de passe respectant la politique de sécurité

Then un compte est créé et un email de vérification est envoyé

And un mot de passe faible ou un email déjà utilisé déclenche une erreur explicite sans créer de compte.

WBS · décomposition hiérarchique orientée livrables

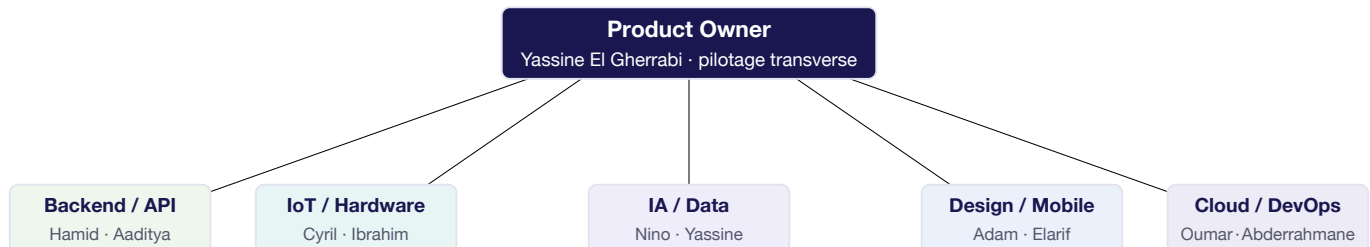
Niveau 0 : Pawrise Care (périmètre total) → Niveau 1 : 11 epics livrables (1 à 11) → Niveau 2 : fonctions (1.1, 1.2...). En italique : exigences (FR/NFR) couvertes.

1 · Onboarding & appairage7 US · MVP 1.1 Compte sécurisé + authentification 1.2 Profil animal (race, âge, poids) 1.3 Appairage BLE du collier (QR code) 1.4 Gestion multi-animaux Exigences : FR12, FR29, FR30, FR33

7 Organisation & compétences

7.1 OBS · organigramme des responsabilités

Dix personnes réparties en cinq pôles de deux, sous le pilotage du Product Owner (Yassine El Gherrabi, qui cumule pilotage produit et contribution IA/Data). L'organigramme (OBS) ci-dessous relie le pilotage à chaque pôle et à ses membres ; le périmètre détaillé de chaque pôle suit.



Elarif est rattaché au pôle Design/Mobile (binôme avec Adam sur l'application) mais intervient en transverse sur le Backend, l'IoT, la CI/CD, le marché et le lien vétérinaire.

Périmètre par pôle

Pôle	Membres	Périmètre
Backend / API	Hamid · Aaditya	API, ingestion, base de données, authentification, Vet Portal
IoT / Hardware	Cyril · Ibrahim	Firmware, capteurs, simulateur, collier
IA / Data	Nino · Yassine	Care Engine, RAG, POCs, pipeline de données
Design / Mobile	Adam · Elarif	UX/UI, app mobile, identité visuelle, étude de marché
Cloud / DevOps	Oumar · Abderrahmane	Cloud, CI/CD, monitoring, sécurité

7.2 Couverture nominative

Chaque domaine est porté par un ou plusieurs responsables nommés, avec un backup identifié pour assurer la continuité.

Domaine	Responsable(s)	Backup
Product Ownership	Yassine	Elarif (transverse, vision produit)
IoT / Hardware	Cyril, Ibrahim	Hamid, Elarif (support embarqué)
Backend / API	Hamid, Aaditya, Elarif	Yassine
IA / Data	Nino, Yassine	Hamid (pipelines Python)
Mobile / Frontend	Adam, Elarif	Hamid, Aaditya
Design / UX	Adam	Elarif (intégration front)
Market / Business	Adam, Elarif	Yassine (PO)
Cloud / DevOps	Oumar, Abderrahmane	Elarif (CI/CD)
Réseau vétérinaire	Elarif, Nino	Binôme (2 contacts vété distincts)

7.3 Matrice de compétences (extrait)

Les **compétences requises** pour concevoir, développer et livrer Pawrise Care (management, embarqué, backend, IA, mobile, design, cloud), et leur **couverture** par chaque membre (auto-évaluation). Chaque compétence critique a au moins un profil confirmé.

Niveaux : ■ expert · ■ intermédiaire · ■ débutant · ■ aucune expérience.

Compétence	Yassine	Elarif	Ibrahim	Cyril	Aaditya	Hamid	Nino	Adam	Oumar	Abderrahmane
Rust	■	■	■	■	■	■	■	■	■	■
Python	■	■	■	■	■	■	■	■	■	■
Microcontrôleurs (ESP32)	■	■	■	■	■	■	■	■	■	■
Firmware C/C++ embarqué	■	■	■	■	■	■	■	■	■	■
IA · NLP / LLM / RAG	■	■	■	■	■	■	■	■	■	■
React / Next.js	■	■	■	■	■	■	■	■	■	■
React Native / Expo	■	■	■	■	■	■	■	■	■	■
Docker / Conteneurisation	■	■	■	■	■	■	■	■	■	■
CI/CD	■	■	■	■	■	■	■	■	■	■
Architecture logicielle	■	■	■	■	■	■	■	■	■	■
UI / UX Design	■	■	■	■	■	■	■	■	■	■
Réseau vétérinaire	■	■	■	■	■	■	■	■	■	■

Lecture de la matrice : forces & points de vigilance

La matrice se lit dans deux sens. **En forces** : plusieurs compétences sont couvertes par au moins deux profils experts (Python, Docker, architecture logicielle, web React/Next) ; combinées aux binômes de pôle et aux backups nominatifs (voir Couverture), elles garantissent qu'aucune brique courante ne dépend d'une seule personne. **En points de vigilance** : quatre compétences critiques ne comptent qu'un seul profil expert. Chacune fait l'objet d'un plan de mitigation explicite.

Compétence critique	Seul expert	Risque & mitigation
IA · NLP / LLM / RAG (Care Engine)	Yassine	Cœur différenciant sur une personne → pipeline LangGraph borné et documenté (ADR), code testé et réappropriable, Nino en appui data.
Électronique embarquée / firmware	Cyril	Le collier dépend d'un expert → simulateur de capteurs en fallback (le reste n'attend pas le matériel), Ibrahim puis Hamid et Elarif en backup embarqué.
Design / UX	Adam	Identité produit sur une personne → Elarif (intermédiaire) prend le relais sur l'intégration front.
Réseau vétérinaire	Elarif	Lien terrain sur une personne → Nino (intermédiaire) constitue un second contact vétérinaire.

Ces quatre points recoupent volontairement des co-fondateurs ou responsables de pôle : le risque est concentré là où l'engagement est le plus fort. Au-delà de ces personnes, la **montée en compétence**

est organisée (Scrum Master tournant, revues de code croisées, documentation d'onboarding) pour diffuser progressivement ces expertises.

7.4 Justification du recrutement

Équipe constituée autour d'un principe : chaque membre couvre un **besoin réel** du projet (IoT, Backend, IA, Mobile, Design, Cloud, Business) tout en étant investi dans la mission. Trois membres sont co-fondateurs et promoteurs du projet (marqués ★). Justification professionnelle, membre par membre :

Membre & rôle	Justification professionnelle
Yassine · PO & Resp. IA ★	3 ans chez Smart Tribune sur NLP/LLM/NER en production → maîtrise du Care Engine, cœur différenciant. Profil architecte (Rust, Python, Kafka, K8s).
Elarif · Resp. Fullstack ★	Fullstack complet (React/Angular/Vue, Python/Java), CI/CD chez Naval Group (GitLab, SonarQube). Intervient en transverse (Backend, IoT, CI/CD). En contact direct avec une vétérinaire partenaire → ancrage terrain.
Cyril · Resp. IoT / Hardware	Le plus expérimenté en électronique embarquée : STM32/ESP32, firmware C/C++, MQTT. Projets persos alignés (BioLink, Centaurus). A déjà résolu les problèmes du collier.
Ibrahim · Resp. IoT ★	Fullstack évoluant vers l'IoT en alternance (capteurs d'irrigation, LoRaWAN) → jonction firmware collier ↔ backend.
Aaditya · Dev Backend / API	Backend orienté API avec sensibilité métier : transformer les données capteurs en valeur. Rigueur et engagement long terme.
Hamid · Dev Fullstack	Fullstack polyvalent (TypeScript/React/Express/Python), gestion VPS et CI/CD. Curiosité embarqué → backup du pôle IoT.
Nino · Resp. Data	Data Engineer en alternance (ETL Python) → pipeline de données du moteur IA. Connaît une étudiante vétérinaire (validation des données).
Adam · Resp. Design & Market	Double profil dev fullstack + designer/commercial. 3 ans de freelance (acquisition client), maîtrise Figma → du wireframe au code (refonte CapFiEurope v2).
Oumar · Resp. Cloud & DevOps	Infrastructure et déploiement : pipelines CI/CD, dev mobile Android. Garant du cycle de livraison (test, déploiement, monitoring).
Abderrahmane · Cloud & DevOps (renfort)	MSc Pro Cloud Epitech (même socle qu'Oumar, Azure). Backend Django/Node, UML/architecture. Arrivé plus tard dans l'équipe : l'architecture ayant évolué vers Kubernetes, le GitOps (Argo CD) et une observabilité complète, un binôme est devenu nécessaire pour épauler Oumar, absorber cette charge accrue et éviter le point de défaillance unique sur le pôle le plus exposé à la montée en charge.

★ co-fondateur et promoteur du projet.

7.5 Compétences non couvertes par l'équipe

Pawrise Care est un **projet de fin d'études (EIP)** : l'équipe conçoit le produit et en livre un **prototype / MVP**, sans budget d'entreprise. Certaines compétences ne sont donc détenues par **aucun** membre. Plutôt que de les passer sous silence, chacune est traitée par une stratégie réaliste à notre échelle : **monter en compétence, s'appuyer sur notre partenaire vétérinaire, utiliser des ressources gratuites et l'encadrement Epitech, ou reporter explicitement** à une éventuelle phase produit (hors périmètre académique).

Compétence manquante	Enjeu pour le projet	Stratégie & mitigation (échelle EIP)
Électronique de série (PCB, miniaturisation)	Un collier industrialisé demande de l'électronique de série ; hors de portée d'une équipe étudiante sans budget.	Hors périmètre académique : on valide toute la chaîne logicielle sur un simulateur de capteurs ; l'industrialisation du matériel est renvoyée à une phase produit.
Base de données séries temporelles	Fort volume de télémétrie à stocker et interroger efficacement ; aucun expert dédié.	Monter en compétence : apprentissage ciblé sur un outil éprouvé (TimescaleDB), qui reste du PostgreSQL familier.
Sécurité applicative	Poser des bases saines (authentification, secrets, mTLS) sans spécialiste sécurité dans l'équipe.	Ressources gratuites & encadrement : checklists OWASP, revues de code entre pairs, cours et intervenants Epitech ; un audit approfondi relèverait d'une phase produit.
Expertise vétérinaire médicale	Corpus et critères d'escalade crédibles ; pas de vétérinaire dans l'équipe (et, par choix, aucun diagnostic produit).	Partenaire : vétérinaire partenaire (contact d'Elarif) et étudiante vétérinaire (via Nino) pour valider le corpus et les seuils d'orientation.
Conformité RGPD / AI Act	Traitement de données et IA encadrés par la loi ; pas de juriste dans l'équipe.	Ressources gratuites : guides et modèles de DPIA de la CNIL (gratuits), principes RGPD appliqués dès la conception ; le PO en est garant.
Commercial / acquisition client	La croissance suppose une force commerciale, absente à ce stade.	Reporter : hors périmètre académique (post-MVP) ; premiers canaux envisagés via partenariats (assurances, cabinets vétérinaires).

8 Matrice RACI

La RACI attribue, pour chaque activité, un rôle à chaque pôle : **R** responsable (produit le livrable), **A** approbateur (garant, valide), **C** consulté, **I** informé.

8.1 Règle appliquée

LA VRAIE RÈGLE RACI

Le seul impératif du standard est : **exactement un A (garant) par activité** et **au moins un R**. Les C et I sont en nombre libre. Un même pôle peut être A et R (noté A/R). Nous présentons deux lectures : une version standard (A distribué par pôle) et une version au format demandé en suivi (un R, un A, un C par activité).

Légende : **A** garant · **R** responsable · **C** consulté · **I** informé.

8.2 Version standard (A distribué)

Chaque pôle est garant de son propre livrable ; le Product Owner reste garant du cadrage et de la gouvernance (RGPD, qualité).

Activité	Pilotage (PO)	Fullstack	IoT	Design/ Mobile	IA/Data	Cloud/ Ops
Cadrage & spécifications	A/R	C	C	C	C	C
API & ingestion données	C	A/R	C	I	C	C
Collier & firmware	C	C	A/R	I	C	I
Application mobile	C	R	I	A/R	C	I
Moteur IA / Care Engine	C	C	I	C	A/R	I
Portail vétérinaire	C	A/R	I	C	C	I
Infra / CI-CD / monitoring	I	C	I	I	I	A/R
RGPD & sécurité données	A	R	C	I	C	C
Tests & qualité	A	R	R	R	R	R

8.3 Version format suiveur (1 R / 1 A / 1 C)

Exactement un R, un A et un C par activité, le reste en I. Plus stricte que le standard, mais toujours valide (un A unique, au moins un R). Le Product Owner est l'approbateur unique.

Activité	Pilotage (PO)	Fullstack	IoT	Design/ Mobile	IA/Data	Cloud/ Ops
Cadrage & spécifications	A	R	I	I	C	I
API & ingestion données	A	R	I	I	I	C
Collier & firmware	A	C	R	I	I	I
Application mobile	A	C	I	R	I	I
Moteur IA / Care Engine	A	C	I	I	R	I
Portail vétérinaire	A	R	I	I	C	I
Infra / CI-CD / monitoring	A	C	I	I	I	R
RGPD & sécurité données	A	R	I	I	I	C
Tests & qualité	A	R	I	I	C	I

Le C unique n'est pas arbitraire : c'est le pôle dont le livrable **dépend le plus directement**, son interface principale. Par exemple, le collier consulte le **Fullstack** (protocole de transmission vers le backend), et le portail vétérinaire consulte l'**IA/Data** (résumés et alertes affichés). Le cadrage initial, transverse par nature, consulte en réalité tous les pôles (voir version standard) ; la version stricte n'en retient qu'un par convention de format.

8.4 Contrôle de cohérence

Les deux matrices respectent des invariants vérifiables, garants de leur validité :

- **Un A par activité** : chaque activité a exactement un garant → aucune décision sans responsable de validation.
- **Au moins un R par activité** : chaque livrable a un pôle qui le produit → aucun angle mort.
- **Chaque pôle est R (ou A/R) au moins une fois** : aucun pôle sans responsabilité propre (Fullstack : API et portail ; IoT : collier ; Design/Mobile : app ; IA/Data : Care Engine ; Cloud/Ops : infra ; PO : cadrage).
- **Gouvernance centralisée** : le PO reste garant (A) du cadrage, de la conformité RGPD et de la qualité, cohérent avec son rôle transverse dans l'OBS.

Les 6 colonnes recoupent l'OBS : Pilotage (PO), Fullstack (backend & portail), IoT (collier), Design/Mobile (app), IA/Data (Care Engine), Cloud/Ops (infra).

9 Planning & méthodologie

9.1 Vue d'ensemble

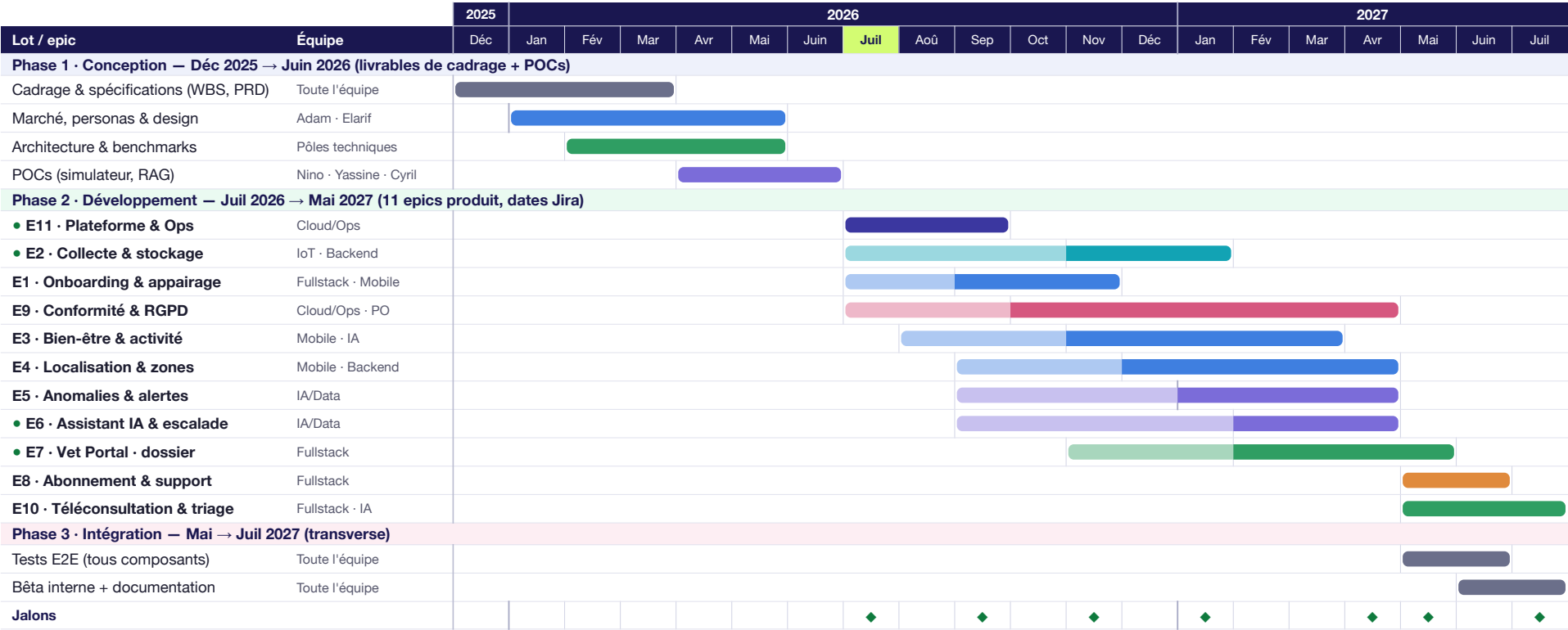
Projet sur 20 mois (décembre 2025 → juillet 2027), en trois phases : **Conception** (livrables pour la keynote de juillet 2026), **Développement** (les 11 epics produit, en parallèle), **Intégration** (assemblage, tests, stabilisation). Le parallélisme est possible car les composants sont volontairement peu couplés ; le portail vétérinaire (E7) est le seul vraiment dépendant (backend + IA), d'où un démarrage plus tardif.

9.2 Diagramme de Gantt

Le diagramme ci-après (page suivante, en paysage) est **aligné sur les 11 epics du WBS** (mêmes codes E1–E11, mêmes couleurs de domaine) avec des **dates issues du backlog Jira**, pour une traçabilité directe WBS → PRD → Jira. Il reflète le travail réel de **5 pôles en parallèle** : aucun epic n'attend qu'un autre soit terminé. Chaque epic démarre par une **préparation** non bloquante (segment clair : design, maquettes / mocks, simulateur, machinerie) puis **consolide** quand sa dépendance réelle est prête (segment plein). Le **chemin critique** (●) et les **jalons** (◆) sont positionnés sur la frise ; la colonne du mois courant est surlignée. Le séquençement est justifié sous le diagramme.

Diagramme de Gantt · Pawrise Care

Barre = epic (couleur = domaine WBS) · consolidation / intégration · préparation (design / mocks / simulateur / machinerie) · chemin critique · jalon · mois courant



Jalons : ♦ Keynote (juil. 2026) · Infra op. (sept.) · Simulateur (nov.) · Backend complet (janv. 2027) · MVP intégré (avr.) · Portail Vétro (mai) · Fin de projet (juil. 2027). E8 et E10 sont post-MVP (démarrage après le MVP d'avril 2027). Détail en table ci-après.

9.3 Jalons

#	Date	Jalon
1	Juil 2026	Keynote (tous les livrables de conception prêts)
2	Sep 2026	Infrastructure opérationnelle (E11)
3	Nov 2026	Simulateur collier (dé-risquage E2)
4	Jan 2027	Backend complet (E1, E2)
5	Avr 2027	MVP intégré (E3–E6, E9 : Care Engine + backend)
6	Mai 2027	Portail vétérinaire livré (E7)
7	Juil 2027	Fin de projet (E8, E10, intégration : produit testé et stabilisé)

9.4 Séquencement & dépendances

Les 5 pôles avancent en parallèle dès juillet ; **aucun pôle n'est bloqué** : chacun démarre la part non bloquante de son epic (design, maquettes / mocks, simulateur, machinerie) et ne **consolide** qu'une fois la dépendance réelle disponible. Quatre dépendances structurent l'ordre :

- **E11 Plateforme (socle)** : tout se déploie sur l'infrastructure ; construite en premier (juil. → sept.), puis exploitée en continu (déploiements, monitoring, RGPD infra).
- **E2 Collecte → features & IA** : la donnée alimente tout. Le **simulateur** (dès juillet) permet aux features (E3, E4) et à l'IA (E5, E6) de démarrer sur données simulées, sans attendre le collier réel.
- **Baseline de données → E5 Anomalies** : détecter une anomalie exige plusieurs mois d'historique par animal (démarrage à froid). La machinerie se prépare dès septembre ; le calibrage sur données réelles vient en décembre.
- **E6 IA + E2 données → E7 Vet Portal** : le portail affiche résumés et alertes. Son échafaudage démarre sur mocks (nov.) ; le câblage aux vraies sorties IA vient en février.

E8 (abonnement) et E10 (téléconsultation) sont **post-MVP** : différés par priorité (non requis pour valider le produit), pas par blocage technique.

9.5 Méthodologie

Scrum adapté (agile itératif). Le périmètre évolue avec les apprentissages (POCs IA, faisabilité hardware, retours du vétérinaire partenaire) : on livre par incréments, on réoriente à chaque sprint. Sprints de 2 semaines, daily / point d'équipe, sprint review, rétrospective. Rôles : Product Owner (Yassine), Scrum Master tournant, équipe par pôles. Outils : Jira (backlog, sprints, story points), Confluence (docs), GitHub (code, PR, CI), Figma (design).

Alternatives écartées : Waterfall (cahier des charges figé, incompatible avec l'incertitude R&D) ; Kanban pur (pas de cadence ni d'engagement de sprint, peu de prévisibilité pour 10 personnes) dont on retient le tableau visuel et la limite de WIP.

Estimation : Planning Poker sur échelle de Fibonacci (1, 2, 3, 5, 8, 13). Au-delà de 13 points, le ticket est découpé.

9.6 Outil de pilotage (Jira)

Le projet est piloté sur un board **Jira** (projet SCRUM, pawrise.atlassian.net) adopté par les **10 membres** : backlog produit, sprints, estimation en story points (Planning Poker) et **assignation nominative** des tâches. Les 11 epics du Gantt y portent leurs dates de début et d'échéance. Confluence héberge la documentation, GitHub le code et les revues (Pull Requests + CI), Figma le design. Le suivi des livraisons s'appuie sur un flux à quatre statuts.

Indicateur	Valeur
Tickets	170+ (epics, tâches, sous-tâches, user stories)
Epics	23 · 12 cadrage & documentation, 11 produit
User stories produit	66, chacune avec critères d'acceptation Given/When/Then
Membres avec tickets assignés	10 / 10
Flux de statuts	À faire → En cours → Revue → Terminé

10 Plan qualité

Stratégie de tests par couche, conventions, workflow Git, CI/CD et onboarding : de quoi produire des livrables fiables et intégrer un nouveau développeur rapidement.

10.1 Definition of Done

Conditions qu'une User Story doit toutes remplir pour être considérée comme terminée :

- Les critères d'acceptation (Given/When/Then) sont tous satisfaits.
- Le code est relu et approuvé par au moins un pair via une Pull Request.
- Les tests de la story sont écrits et passent : **unitaires et fonctionnels obligatoires** (intégration et E2E si la story touche un flux critique).
- La couverture de code atteint le seuil fixé (mesurée en CI, merge bloqué sinon).
- La CI est verte : format, lint, tests et audit automatiques réussis (le merge est bloqué sinon).
- La documentation utile est à jour (README du service, doc d'API, Confluence si besoin).
- Aucune régression : un bug corrigé est couvert par un test de non-régression.

RÈGLE D'OR PRODUIT

L'IA n'émet jamais de diagnostic. Les tests du Care Engine vérifient que toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

10.2 Stratégie de tests par couche

Deux niveaux sont **obligatoires** pour chaque story (inscrits dans la Definition of Done) : tests **unitaires** (logique isolée) et **fonctionnels** (comportement d'un module ou d'un endpoint). Les **flux critiques** sont en plus couverts par des tests **E2E / smoke** : onboarding et appairage, ingestion jusqu'à l'alerte, escalade IA vers vétérinaire, paiement. La couverture est mesurée et contrôlée en CI (seuil ci-dessous).

Couche	Outils	Cible
Firmware collier (Rust no_std)	Tests unitaires embarqués + simulateur de capteurs	Logique de collecte/encodage validée hors matériel via le simulateur.
Services backend (Rust)	Tests unitaires (cargo nextest) + tests d'intégration API sur services réels (testcontainers)	Logique métier et endpoints couverts ; tests d'abus obligatoires sur l'authentification.
Moteur IA / Care Engine (Python)	pytest + jeux d'évaluation RAG + garde-fous anti-diagnostic	Non-régression des réponses et escalade systématique en cas de doute.
App mobile (Kotlin / SwiftUI)	Tests unitaires natifs + tests d'UI critiques	Onboarding, appairage et alertes vérifiés.
Bout en bout (E2E / smoke)	Scénarios end-to-end sur les flux critiques (phase d'intégration)	Collier → backend → IA → app/portail validés ensemble.
Performance / charge	Tests de charge (k6 / Locust) sur les endpoints critiques	Temps de réponse p95 et montée en charge validés avant la mise en production (NFR6, NFR7).

10.3 Couverture de code : objectif & seuil

Chaque story livre ses tests ; la **couverture de ligne** est mesurée en CI et **bloque le merge** sous le seuil. Nous nous appuyons sur les paliers de Google ([Code Coverage Best Practices](#)) : **60 % « acceptable », 75 % « commendable », 90 % « exemplary »**.

- **Objectif : 90 %** (palier « exemplary ») sur le **code métier critique** : garde-fous du Care Engine, ingestion, authentification, paiement.
- **Plancher bloquant en CI : 80 %** sur l'ensemble du code (via `cargo llvm-cov --fail-under-lines`), au-dessus du palier « commendable ».
- **Pas de course au 100 %** : au-delà de 90 %, le gain devient logarithmique (beaucoup d'effort pour un risque résiduel faible) ; Google déconseille explicitement les objectifs projet au-dessus de 90 %.
- Ce qui compte autant que le chiffre : l'**analyse humaine des lignes non couvertes** (le risque résiduel est-il acceptable ?), pas la métrique brute.

10.4 Conventions de code

- Linting/formatage automatiques par langage : `rustfmt` + `clippy` (Rust), `ruff/black` (Python), `ktlint` (Kotlin), `ESLint` + `Prettier` (TS/Next).
- Nommage cohérent par langage (`snake_case` Rust/Python, `camelCase` Kotlin/TS) et noms descriptifs.
- Toute fonctionnalité passe par une Pull Request relue par au moins un pair avant merge.
- Pas de secret en clair dans le code : variables d'environnement + secrets chiffrés (SOPS).

10.5 Workflow Git

Branche	Rôle
main	Branche stable et déployable. Protégée : merge uniquement via PR validée.
develop	Intégration continue des fonctionnalités avant stabilisation.
feat/<scope>	Une fonctionnalité = une branche, rattachée à un ticket Jira.
fix/<scope>	Correction de bug isolée et traçable.

Commits conventionnels (`feat:`, `fix:`, `docs:`, `chore:...`) référençant la clé Jira. **Pull Request** : description claire, lien Jira, CI verte et revue d'au moins un pair avant merge.

10.6 Intégration continue (CI)

Architecture quasi **full-Rust** : la CI (GitHub Actions) est centrée sur l'outillage Cargo. À chaque Pull Request, **tout gate est bloquant** (merge impossible si rouge) :

Étape	Commande / rôle
Format	<code>cargo fmt --all --check</code> : style uniforme, non négociable.
Tri des dépendances	<code>cargo sort --workspace --check</code> : Cargo.toml ordonnés.
Lint	<code>cargo clippy --workspace --all-targets -- -D warnings</code> : zéro warning toléré.
Dépendances inutilisées	<code>cargo machete</code> : pas de dette de dépendances.
Tests + couverture	<code>cargo llvm-cov nextest --workspace --fail-under-lines 80</code> : tests rapides (nextest) + seuil de couverture.
Audit de sécurité	<code>cargo audit</code> : CVE des dépendances (base RustSec).

Les tests d'intégration tournent sur des **services réels éphémères** (bases, brokers) via `testcontainers`. Les autres stacks ont leurs propres gates : **Python** (Care Engine) `ruff` + `pytest` + jeux d'évaluation ; **mobile** `SwiftLint` / `ktlint` (`--strict`) + tests natifs avec couverture. Un cache (`Swatinem/rust-cache`) accélère la CI. La branche `main` est **protégée** : merge uniquement via PR verte et relue.

10.7 Sécurité des dépendances & supply chain

- **Dependabot** : mises à jour hebdomadaires et groupées (crates Cargo, GitHub Actions, dépendances mobile), ouvertes en PR automatiques passant par la même CI.
- **cargo audit** (RustSec) à chaque PR + job de sécurité dédié ; **cargo deny** prévu pour le contrôle des licences.
- **Secrets chiffrés** (SOPS + age), jamais en clair dans le dépôt ; scan de secrets sur les PR.
- **Protection de branche** : revue d'au moins un pair obligatoire, historique linéaire, CI verte requise.

10.8 Déploiement (CD)

- Build et publication d'**images conteneurisées** (Docker) versionnées à chaque release.
- **Trois environnements** : develop → **staging** (validation intégrée) → **production**, promus par merge ; un canal **hotfix** dédié pour les correctifs urgents en production.
- Déploiement **GitOps** : Helm + Argo CD synchronisent le cluster Kubernetes depuis Git (source de vérité).
- Observabilité OpenTelemetry → Prometheus / Loki / Tempo → Grafana ; rollback par simple revert Git.

10.9 Onboarding développeur

- Accès dépôt GitHub, projet Jira et espace Confluence dès l'arrivée.
- Lecture d'entrée : ce plan qualité, le Git Workflow et l'architecture globale.
- Mise en route locale : clone, installation, lancement en local (README de chaque service).
- Première contribution : un ticket « bonne première tâche », une PR suivant les conventions.

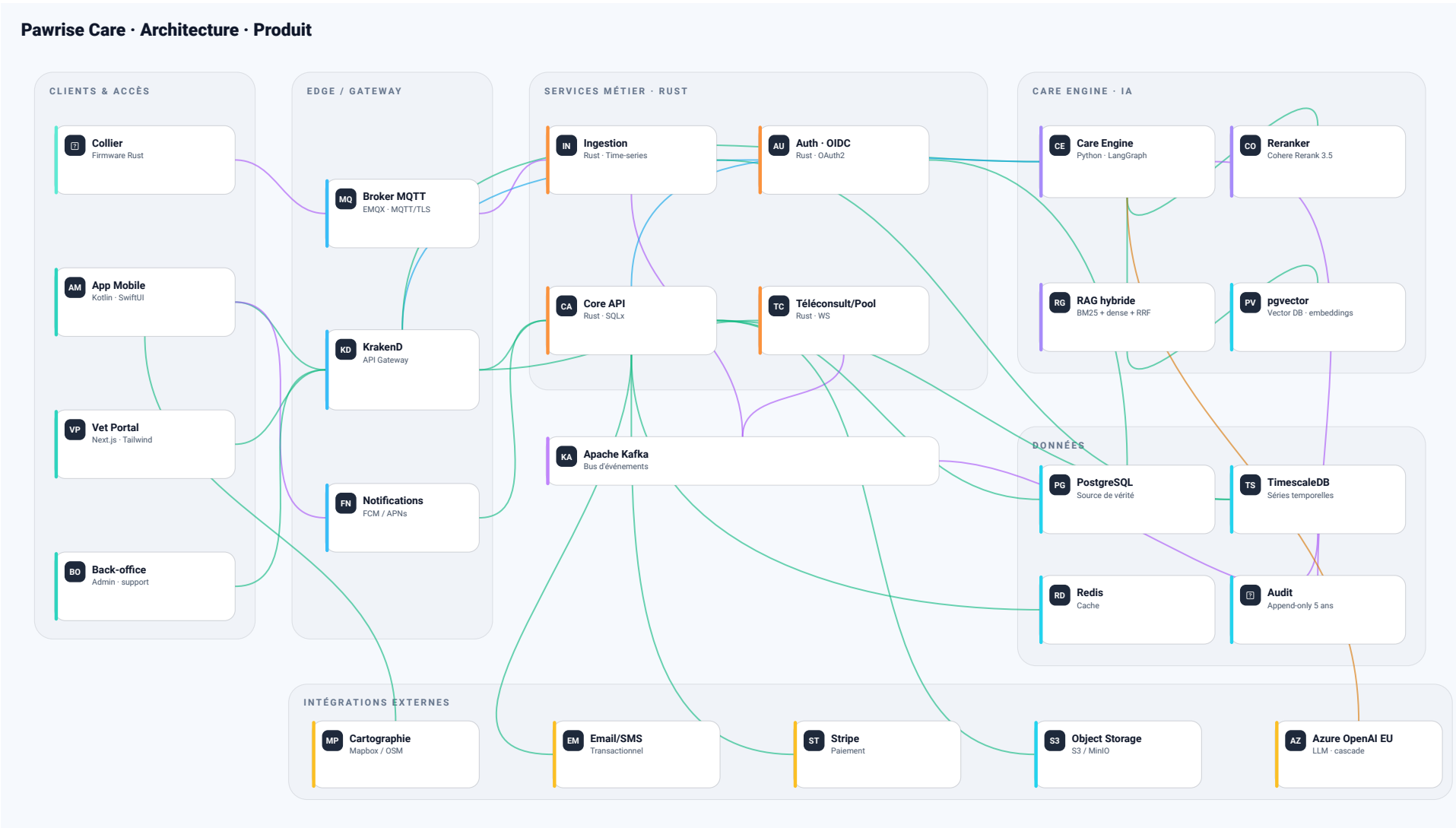
11 Architecture technique

11.1 Vue d'ensemble

L'architecture suit **quatre principes directeurs**, chacun assumé et justifié plus bas :

1. **Monolithe modulaire d'abord, macroservices ensuite** : un socle unique (API + Care Engine) pour la vélocité d'une équipe de 10, découpé en services indépendants seulement aux points chauds réels.
2. **Architecture événementielle** : les services communiquent via un bus (Kafka) pour se découpler et ne perdre aucun événement (ingestion, projections, alertes, audit, file de téléconsultation).
3. **IA en cage** : le Care Engine est un pipeline borné qui explique et oriente, sans jamais diagnostiquer ni écrire directement dans les données métier.
4. **Souveraineté et coût maîtrisé** : Rust économe, une seule famille de bases (PostgreSQL), hébergement UE auto-géré.

Flux : — synchrone (aller-retour) · — événement (sens unique) · — contrôle · — externe.



Plateforme & Ops (transverse) : Kubernetes auto-géré (Hetzner, Terraform) · GitOps (Argo CD) · observabilité OpenTelemetry → Prometheus / Loki / Tempo / Grafana · secrets chiffrés (SOPS). Détail en partie Cloud & infrastructure. Diagramme exporté du cockpit interactif : version complète et navigable sur pawrise-care.com.

Lecture par zone

- **Clients & accès** : une **app mobile native** (bien-être, localisation, alertes, chat IA, appairage BLE), un **Vet Portal web** (historique structuré + PDF normalisé) et une console **Admin**. Tout le trafic applicatif passe par **KrakenD**, une passerelle unique : un seul point d'entrée à sécuriser, qui centralise authentification, limitation de débit et routage.
- **Entrée IoT** : le **collier** parle en **MQTT sur TLS** (certificat par appareil) à un broker **EMQX**, séparé de la passerelle HTTP. MQTT est le standard IoT : pub/sub léger, QoS et mode hors-ligne gratuits, adaptés à une batterie et un réseau cellulaire instable.
- **Services métier (Rust)** : **OIDC** (identité et droits owner / vétérinaire), **Core API** (comptes, animaux, alertes), **Ingestion** (normalisation de la télémétrie), **véto & abonnement**. Ils forment d'abord un **monolithe modulaire** : on n'extrait un macroservice que lorsqu'un besoin réel (charge, équipe) le justifie.
- **Bus d'événements (Kafka)** : la colonne vertébrale. Les services **publient** des événements ; les consommateurs (projections, Care Engine, audit, file de téléconsultation) s'y **abonnent**. Résultat : services découplés et **aucun événement perdu**.
- **IA · Care Engine (Python)** : un pipeline **LangGraph borné** qui consomme le **RAG** (pgvector) et le LLM (**Azure OpenAI UE** + reranker Cohere). Il **ne peut pas écrire** dans les données métier : il produit une orientation et, si besoin, **escalade** vers le vétérinaire via une interface contrôlée.
- **Données** : une seule famille **PostgreSQL** : transactionnel (PostgreSQL), séries temporelles de télémétrie (**TimescaleDB**) et embeddings du corpus (**pgvector**), plus **Redis** pour le cache et les sessions. Un seul moteur à opérer, sauvegarder et sécuriser.

L'IA EN CAGE · RÈGLE D'OR

Le Care Engine n'a **aucun accès en écriture** au métier et suit un pipeline déterministe : il explique, oriente et escalade, **jamais il ne diagnostique** (Code rural, art. L243-1). Cette contrainte est portée par l'architecture elle-même, pas seulement par le modèle, ce qui la rend juridiquement défendable (ADR-001).

11.2 Choix techniques justifiés

Chaque décision structurante : besoin, alternatives écartées, décision, et trade-off assumé.

Choix	Alternatives écartées	Décision & trade-off
Rust (services métier)	Node, Python, Go, C#	Perf + sûreté mémoire, empreinte minimale. Coût dev ~2,5× assumé (mitigé : monolithe-first, Python pour l'IA, réintroduction ciblée).
MQTT / EMQX (entrée IoT)	HTTP, CoAP, passerelle maison	Standard IoT : pub/sub léger, QoS, offline gratuits. Un composant de plus à opérer.
Apache Kafka (bus)	RabbitMQ, appels directs	Débit, rétention, découplage. Plus gourmand en RAM (dimensionne le cluster).
PostgreSQL + TimescaleDB Mongo + Influx + Qdrant séparés + pgvector		Une seule famille : transactionnel, séries temporelles, vectoriel. Bascule Qdrant possible.
Pipeline LangGraph borné	Agent LLM autonome	Chaque étape testable, responsabilité juridique portée par le pipeline (ADR-001).
Azure OpenAI EU	LLM hors UE, modèle unique	Data residency UE, ~0,03-0,05 €/conv. Dépendance Microsoft mitigée (interface LLMPProvider).

Choix	Alternatives écartées	Décision & trade-off
+ cascade + Cohere		
Kubernetes auto-géré + GitOps	PaaS, managé d'emblée	Économique et souverain ; plus d'ops, mitigé par l'IaC. Managé EU au scale.
OIDC maison	Auth0, Keycloak, Cognito	Souveraineté + défi formateur ; coûteux (~4-6 mois-homme), revue de sécurité obligatoire.

12 Cloud & infrastructure

Une infrastructure **souveraine** (UE / RGPD), **maîtrisée en coût** et **prête à scaler** : Hetzner auto-géré via Terraform aujourd'hui, cloud managé européen le jour où la charge le justifie. Décrite en code, déployée en GitOps. Cette partie justifie le dimensionnement (à partir de l'architecture), le choix d'hébergement (bare metal / Hetzner / managé / hyperscaler) et l'exigence de souveraineté.

12.1 Principes directeurs

- **Souveraineté & RGPD** : infrastructure et données de santé en UE ; Azure seulement pour le modèle de langage (région UE).
- **Infrastructure as Code** : toute l'infra en Terraform, un cluster se recrée à l'identique en une commande.
- **GitOps** : Git seule source de vérité ; Argo CD synchronise le cluster, rollback par revert.
- **Maîtrise des coûts** : lignes ARM, environnements de dev éphémères, auto-géré moins cher qu'un managé à capacités égales.
- **Observabilité** : chaque service tracé (OpenTelemetry) ; métriques, logs, traces centralisés dans Grafana.

12.2 Dimensionnement (issu de l'architecture)

Les besoins ne sont pas choisis au hasard : ils découlent des charges de travail décrites en partie Architecture (services Rust, bases de données, bus d'événements, broker MQTT, moteur IA, observabilité). Estimation à faible charge, pour un lancement :

Charge de travail	vCPU	RAM
Services métier Rust (Core API, ingestion, auth, portail...)	~2 à 3	~3 Go
Bases de données (PostgreSQL + TimescaleDB + pgvector)	~2	~6 Go
Bus d'événements (Kafka)	~1 à 2	~4 Go
Broker MQTT (EMQX)	~1	~2 Go
Care Engine (pipeline IA ; le LLM est déporté sur Azure)	~1 à 2	~3 Go
Observabilité (Prometheus / Loki / Tempo / Grafana)	~1 à 2	~4 Go
Gateway, ingress, Argo CD, PKI	~1	~2 Go
Total (avec marge)	≈ 16 vCPU	≈ 32 Go

Rust et l'absence de LLM local (déporté sur Azure) gardent l'empreinte basse. La cible ≈ 16 vCPU / 32 Go correspond à **2× CAX31** (2 × 8 vCPU / 16 Go) en workers, plus **1× CAX21** en control-plane. On monte en charge simplement en ajoutant des workers.

12.3 Comparaison des options d'hébergement

À capacité équivalente (≈ 16 vCPU / 32 Go + bases de données, load balancer, sauvegardes et IA, faible trafic) :

Option	Coût annuel*	Souveraineté UE	Exploitation	Lock-in
Bare metal (serveur dédié)	~1 000 à 1 500 €	Oui	Élevée (matériel, pannes, redondance à doubler)	Aucun
Hetzner Cloud auto-géré (retenu)	≈ 1 600 €	Oui	Moyenne (K8s, mais tout en IaC)	Aucun (Terraform portable)
Cloud managé EU (Scaleway / OVH)	~3 000 à 3 700 €	Oui	Faible (control-plane managé)	Modéré (fournisseur)
Hyperscaler (AWS / Azure / GCP)	~6 000 à 10 000 €	Partielle (Cloud Act US)	Faible	Fort

* Ordres de grandeur, faible trafic et usage IA modéré. Prix serveurs : tarifs publics Hetzner juin 2026 (Cloud CAX et dédiés AX).

Le **bare metal** est le moins cher en compute brut mais impose l'exploitation matérielle (remplacement de disque, redondance physique à acheter en double) et n'offre aucune élasticité : incompatible avec des environnements de dev éphémères. L'**hyperscaler** est confortable mais 4 à 6× plus cher, avec un fort lock-in et une souveraineté seulement partielle (voir ci-dessous). Le **cloud managé européen** est la bonne cible au scale. Pour la phase actuelle, **Hetzner Cloud auto-géré** offre le meilleur compromis coût / souveraineté / flexibilité.

12.4 Souveraineté : pourquoi l'UE

Les données de santé animale, rattachées à un propriétaire identifié, sont des données personnelles au sens du RGPD. Un hébergeur européen évite l'exposition au **Cloud Act** américain, qui peut contraindre un fournisseur US à communiquer des données même hébergées dans une région UE. D'où le choix d'un acteur européen (Hetzner) pour toute l'infrastructure ; le seul service non européen, le grand modèle de langage, passe par **Azure OpenAI en région UE**, avec résidence des données et sans réutilisation pour l'entraînement (voir partie IA).

DÉCISION

Cluster Kubernetes sur **VPS Cloud Hetzner (lignes ARM CAX)**, auto-géré et provisionné par Terraform. Depuis la hausse Hetzner du 15 juin 2026 (jusqu'à +176 % sur l'AMD, +30 % seulement sur l'ARM), la ligne ARM est la plus rentable et suffit à notre charge. Bare metal écarté (exploitation matérielle) ; cloud managé européen gardé pour le passage à l'échelle.

12.5 Stratégie en deux temps

Maintenant (dev → keynote → premiers utilisateurs) : cluster Kubernetes provisionné par Terraform (control-plane CAX21 + 2 workers CAX31), déploiement GitOps, secrets chiffrés, PKI interne, souveraineté UE.

Au scale (traction réelle) : Cloud managé EU, Scaleway Kapsule (control-plane managé gratuit, PostgreSQL managé, Object Storage, GPU à la demande), sans lock-in hyperscaler. Alternative : OVHcloud Managed Kubernetes. Azure AKS gardé pour un besoin multi-région ou un SLA entreprise.

12.6 La stack d'infrastructure

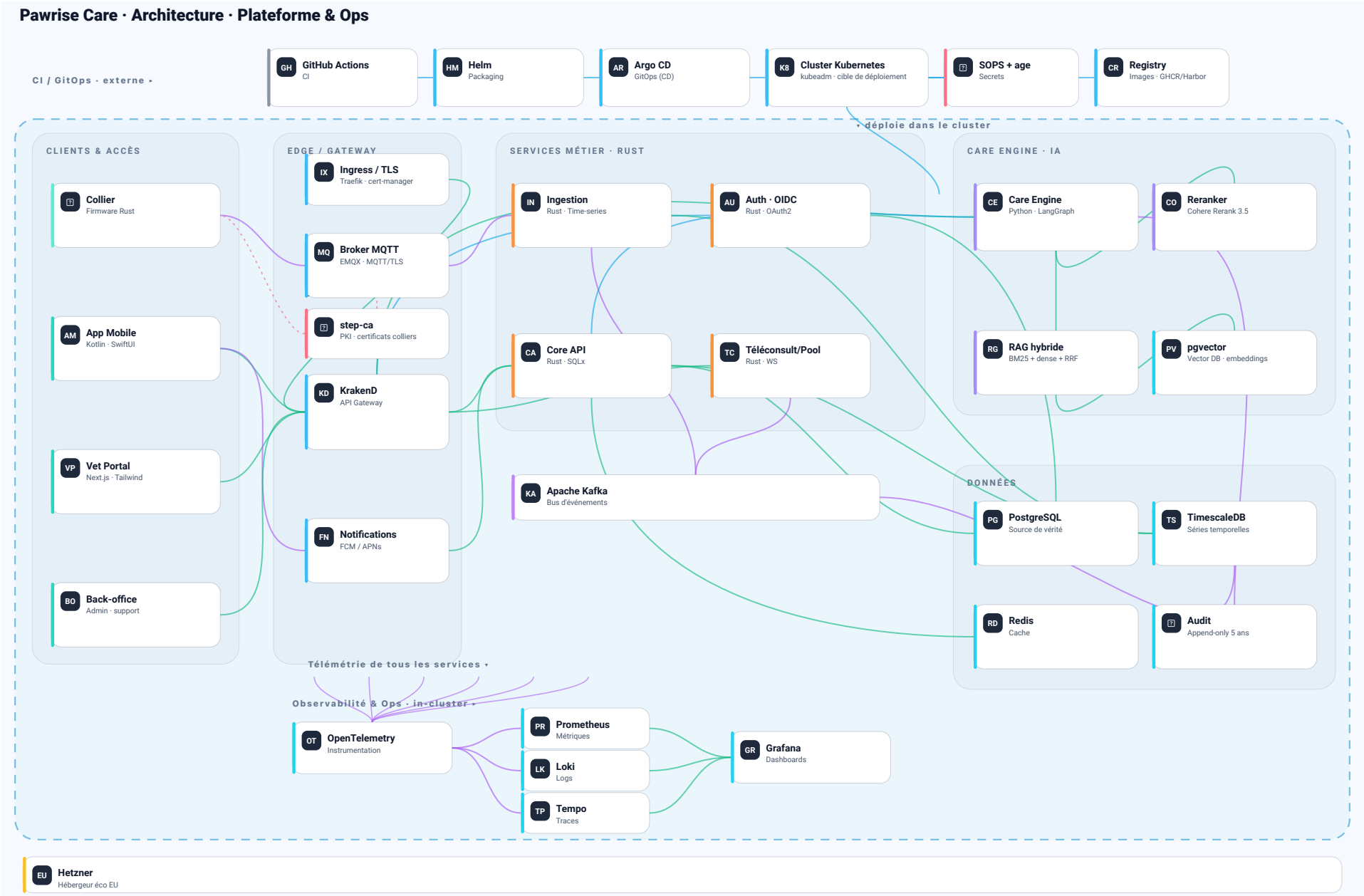
Techno	Rôle	Pourquoi
Terraform	Provisionnement	Décrit et crée toute l'infra Hetzner de façon reproductible ; détruire/recréer un environnement à la demande.

Techno	Rôle	Pourquoi
Kubernetes	Orchestration	Fait tourner et auto-répare la vingtaine de services ; redémarrage, placement, montée en charge.
Helm	Packaging	Chaque service en chart versionné et paramétrable (dev/prod).
Argo CD	Déploiement GitOps	Déploie depuis Git ; l'état du cluster suit le dépôt, rollback par revert.
SOPS + age	Secrets	Secrets chiffrés dans Git, déchiffrés uniquement dans le cluster.
step-ca	PKI interne	Autorité de certification : chaque collier et service s'authentifie en mTLS.
OTel + Prometheus/Loki/Tempo/Grafana	Observabilité	Métriques, logs et traces unifiés.
KrakenD + Traefik / cert-manager	Gateway & Ingress	Agrégation d'API, entrée HTTP et TLS automatique.

12.7 Vue plateforme : topologie de déploiement

Ce schéma (exporté du cockpit interactif) situe chaque brique d'infrastructure autour du cluster : la chaîne **CI/CD externe** (GitHub Actions → Helm → Argo CD → cluster), l'**enceinte Kubernetes** auto-gérée, la **PKI interne** (step-ca) qui authentifie colliers et services, l'**observabilité in-cluster** (OpenTelemetry → Prometheus / Loki / Tempo / Grafana) et l'**hôte Hetzner** qui porte l'ensemble.

_____ contrôle / déploiement · _____ télémétrie · _____ synchrone. Enceinte pointillée = cluster Kubernetes ; socle = hôte Hetzner.



12.8 Maîtrise des coûts en développement

Un serveur arrêté reste facturé chez Hetzner : « éteindre » n'économise rien. Le bon mécanisme s'appuie sur l'laC :

- Facturation horaire avec plafond mensuel : un serveur qui n'existe que quelques heures par jour est facturé au prorata.
- Environnements de dev éphémères : terraform destroy le soir et le week-end, terraform apply le matin.
- Un cluster de dev actif 40 h/semaine (au lieu de 168) coûte 25 % du plafond, soit près de 70 % d'économie sur le compute de dev.
- Données préservées sur volumes persistants (0,057 €/Go/mois) ou snapshots ; seul le compute est recréé.

13 IoT · le collier connecté

Le collier est le cœur produit : capteurs embarqués, connectivité cellulaire basse consommation et firmware Rust, pensés pour tenir sur batterie, fonctionner en zone blanche et transmettre en sécurité.

12 à 15 j

autonomie cible

IP67

poussière + immersion

-20 à +60°C

plage de température

2000 mAh

batterie Li-Po

13.1 Les défis à résoudre

- **Autonomie énergétique** · cible 12 à 15 jours ; GPS, modem et capteurs sont énergivores, d'où veille profonde, capteurs à la demande, transmission par batch.
- **Zones blanches** · le cas critique (fugue) arrive souvent sans couverture ; mode dégradé : stockage local puis rattrapage (store and forward).
- **Précision de localisation** · GPS qui dérive et premier fix long, accélérés par l'A-GPS et une fréquence adaptée au contexte.
- **Robustesse** · chocs, eau, température : certification IP67 et composants qualifiés -20 à +60°C, gestion thermique au contact de la peau.
- **Sécurité des données** · un appareil qui transmet la position doit être inviolable : authentification par certificat et chiffrement de bout en bout.

13.2 Choix du microcontrôleur : nRF9160 (SiP)

UN SEUL SILICIUM, PLUS D'AT COMMANDS

Le nRF9160 est un **SiP** qui intègre le microcontrôleur (ARM Cortex-M33) et le modem LTE-M/NB-IoT dans la même puce. Alternative écartée : un MCU plus un modem externe (ESP32-S3 + BG95, ou LilyGO T-SIM7080G), qui oblige le firmware à piloter le modem par **AT commands** (protocole textuel hérité des années 1980 : chaînes à construire, réponses fragiles à parser, timeouts empiriques, 250+ pages de documentation). Avec le nRF9160, la connectivité est gérée par la bibliothèque modem Nordic (LTE Link Control), exposée à notre firmware **Rust** via la crate `nrf-modem` : plus d'AT commands. Son **TrustZone** sert de secure element pour la clé privée.

13.3 Composants embarqués

Composant	Choix	Rôle
MCU + modem (SiP)	nRF9160-SICA	Exécute le firmware Rust et la machine à états ; modem LTE-M/NB-IoT intégré ; TrustZone = secure element.
GNSS dédié	u-blox ZOE-M8Q	Localisation multi-constellation (GPS/GLONASS/Galileo/BeiDou), plus précise que le GNSS intégré seul.
Centrale inertielle	LSM6DSOX (ST)	Accéléromètre + gyroscope avec Machine Learning Core embarqué : activité, immobilité, chocs.
Température	TMP117 (TI)	Température corporelle, précision $\pm 0,1^\circ\text{C}$.
Fréquence cardiaque	1 option parmi 4 (axe R&D)	Rythme cardiaque (PPG optique ou piézo) ; indicateur de tendance, non médical.
Mémoire flash	W25Q128 (16 Mo)	Tampon local store-and-forward (~72 h).
PMU + régulateur	BQ25895 + TLV62585	Charge Li-Po, fuel gauge I ² C, régulation 3,3 V, protections.
Antennes	Taoglas FPC (LTE-M + GNSS)	Antennes flexibles pour la cellulaire et le GNSS.

Composant	Choix	Rôle
Batterie	Li-Po 2000 mAh (103450)	Alimentation autonome, autonomie cible 12 à 15 jours.

13.4 Capteurs : choix justifiés

- **Activité (IMU)** → **LSM6DSOX** : très basse consommation, Machine Learning Core embarqué (détection de mouvement) qui décharge le MCU.
- **Température** → **TMP117** : précision médicale ($\pm 0,1^\circ\text{C}$), idéale pour détecter une fièvre.
- **Fréquence cardiaque** → **axe R&D (décision Go/No-Go fin du mois 6)** : quatre candidats testés sur chien, MAX30102 (PPG rouge/IR), APDS-9960 (PPG vert 530 nm), MAXM86161 (PPG 3 longueurs d'onde) et un capteur piézocéramique (vibration mécanique). Le PPG optique est calibré pour l'humain : sa fiabilité sur un animal à fourrure et en mouvement doit être validée. En production, une seule option est retenue, ou un emplacement non peuplé (DNP) si No-Go. Les données restent des indicateurs de tendance, non médicaux.

13.5 Réseau & sécurité

Réseau : LTE-M plutôt que NB-IoT (garde la mobilité et une latence faible, indispensables pour suivre un animal), NB-IoT en repli ; antennes flexibles Taoglas FPC (LTE-M et GNSS) ; transport MQTT sur TLS (broker EMQX : pub/sub léger, QoS, offline gratuits) ; SIM IoT 1NCE (500 Mo / 10 ans, roaming 165 pays) ; mode dégradé store-and-forward.

Sécurité : PKI X.509 (Root CA → CA intermédiaire signant colliers et serveur, via step-ca) ; authentification mutuelle mTLS (un appareil non enregistré est refusé, même firmware reverse-engineered) ; provisioning en usine (device_id + certificat, clé dans le TrustZone) ; activation par appairage BLE (QR code).

13.6 Logique embarquée : machine à états

Le firmware Rust adapte acquisitions et transmission selon l'état de l'animal et du réseau : **BOOT** (démarrage), **PAIRING** (appairage BLE), **REST** (repos, sobre), **ACTIVE** (activité), **ESCAPE** (sortie de zone, GPS accéléré), **LIVE** (temps réel à la demande), **NO_NETWORK** (zone blanche, stockage local), **LOW_POWER** (batterie faible, services réduits).

13.7 Du prototype au produit

Phase de développement : kit **nRF9160-DK** + shields/breakouts I²C sans soudure (Qwiic / STEMMA QT), pour valider firmware et capteurs, ≈ 228 € (≈ 332 € avec la R&D fréquence cardiaque à 4 candidats).

Produit final : PCB 4 couches custom, composants nus (SMD/CMS) assemblés chez JLCPCB, boîtier PC/ABS injecté IP67 (70×46×20 mm), antennes FPC. ≈ 131 € en prototype unitaire, ≈ 66 €/u en série de 1000. Détail chiffré en partie Modèle économique & budget.

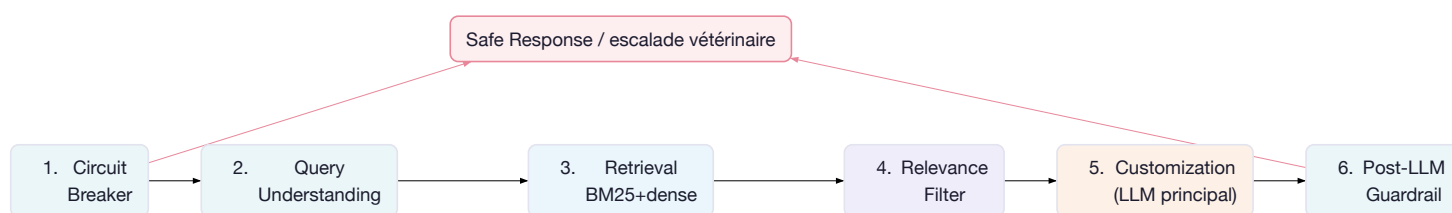
14 IA · le Care Engine

L'assistant conversationnel contextualise l'état d'un chien à partir des données du collier, répond sur la base d'un corpus vétérinaire validé, et organise un handoff structuré vers un vétérinaire quand c'est pertinent.

RÈGLE D'OR

L'assistant n'établit jamais de diagnostic médical (Code rural, article L243-1) : il explique, oriente, et invite à consulter. Architecture : un pipeline LangGraph déterministe borné à 6 nœuds, déployé sur Azure OpenAI Europe ; la mémoire long-terme est déléguée au backend via tool calls.

14.1 Pipeline LangGraph (6 nœuds)



Pipeline déterministe et borné : chaque étape est testable et journalisée (audit append-only, rétention 5 ans). Le LLM est cantonné à sa tâche.

14.2 Garde-fous (défense en profondeur, 3 couches)

- **Couche 1 · pré-LLM (Circuit Breaker)** : classifier d'intention ; tout ce qui n'est pas « clean » est dérouter vers une Safe Response ou une escalade, sans atteindre le LLM principal.
- **Couche 2 · raisonnement** : system prompt strict + whitelist de tools + obligation de citer le corpus pour tout claim médical.
- **Couche 3 · post-LLM** : détection « diagnostic-like », contrôle d'ancrage sur les sources, déclenchement forcé de l'escalade selon des règles explicites.

14.3 Périmètre

Ce qu'on fait : Q&A santé/comportement (corpus + télémétrie), contextualisation d'une alerte collier, handoff vétérinaire structuré (PDF + JSON), refus systématique du diagnostic, audit complet.

Ce qu'on ne fait jamais : diagnostic explicite, prescription/posologie, gestion d'urgences vitales (redirection urgentiste), conseils de dressage, conseils nutritionnels précis.

14.4 Coût des modèles (cascade « un modèle par nœud »)

Chaque nœud utilise le modèle le moins cher qui fait le travail ; seul le nœud de génération mobilise le gros modèle.

Nœud	Modèle	Prix (entrée / sortie, 1M tokens)
1 · Circuit Breaker	mini · GPT-5-nano	0,05 \$ / 0,40 \$
2 · Query Understanding	mini · GPT-5-nano	0,05 \$ / 0,40 \$
3 · Retrieval	embeddings text-embedding-3-large + BM25	0,13 \$ / 0
4 · Relevance Filter	Cohere Rerank 3.5 multilingue	2 \$ / 1 000 recherches
5 · Customization	principal · GPT-5 (ou GPT-4.1)	~1,25 à 2,50 \$ / ~10 \$
6 · Post-Guardrail	mini · GPT-5-nano	0,05 \$ / 0,40 \$

COÛT PAR CONVERSATION ≈ 0,03 À 0,05 €

Répartition : génération 85 %, reranking 10 %, nœuds mini 2 %, embeddings < 0,5 %. La cascade économise 30 à 35 % vs tout passer par le gros modèle. Ce coût valide la cible de la spec (moins de 0,05 €/conversation). Alternative souveraine prévue (ADR-002) : Mistral (UE), coût équivalent, données 100 % UE. Prix éditeurs en dollars, parité dollar/euro retenue.

14.5 Décisions d'architecture (ADR)

ADR	Décision
001 · Pipeline borné	6 nœuds déterministes, pas d'agent autonome : responsabilité juridique portée par le pipeline.
002 · Azure OpenAI EU + cascade	Data residency UE, pas d'entraînement sur les données ; interface LLMProvider (basculer possible).
003 · Reranker dédié	Cohere Rerank 3.5 : latence ~10× meilleure qu'un filtre LLM, meilleure pertinence.
004 · Mémoire = backend	Pas de fact store dans le chatbot ; tool calls vers le Core API (source de vérité unique).
005 · Sortie JSON structurée	Le front affiche les actions sans parser du markdown fragile.
006 · RGPD progressif	MVP : masquage regex des PII ; V1 : Presidio + dictionnaire vétérinaire.
007 · Garde-fous 3 couches	Défense en profondeur : l'échec d'une couche n'expose pas le système.
008 · Retrieval hybride	BM25 (termes exacts) + dense (similarité), fusion RRF.
009 · Query Understanding	Reformulation en termes vétérinaires + tool télémétrie avant le RAG.
010 · Audit append-only	Log immuable de chaque conversation, rétention 5 ans : défense juridique et base d'eval.

14.6 Objectifs de succès (KPIs MVP)

0 faux diagnostic sur le golden set · ≥ 95 % de citation quand claim médical · 5 à 15 % de taux d'escalade approprié · réponse complète p95 < 8 s · recall@5 ≥ 0,85 · détection jailbreak ≥ 95 %.

15 Gestion des risques

La maîtrise des risques repose sur deux analyses complémentaires : une **AMDEC** centrée sur le produit (modes de défaillance techniques) et une **Risk Map** centrée sur le projet (probabilité contre impact).

15.1 AMDEC produit

L'AMDEC évalue chaque mode de défaillance par son indice de priorité de risque **IPR = G × O × D** (Gravité × Occurrence × Détection, chaque facteur de 1 à 10). Les modes sont triés par IPR décroissant : la priorisation se lit de haut en bas.

Code	Sous-système	Mode de défaillance	G	O	D	IPR	Niveau
H03	Capteur température / FC	Mesures erronées (faux positifs physiologiques)	7	7	6	294	Critique
I01	Analyse comportementale	Faux négatif : anomalie non détectée	9	5	6	270	Critique
I03	Chat RAG vétérinaire	Réponse médicalement incorrecte du LLM	9	5	5	225	Critique
E01	Déclenchement automatique	Escalade non déclenchée pour un cas urgent	10	4	5	200	Critique
I02	Analyse comportementale	Faux positif : alerte infondée	5	7	5	175	Critique
H01	Batterie	Décharge rapide / autonomie insuffisante	7	6	4	168	Critique
E02	Portail vétérinaire	Vétérinaire partenaire indisponible	8	5	4	160	Critique
T03	GPS WebSocket	Latence excessive du GPS en temps réel	6	5	5	150	Critique
H04	Communication BLE	Perte de synchronisation smartphone ↔ collier	5	7	4	140	Critique
H02	Capteurs	Défaillance du capteur accéléromètre	6	4	5	120	Critique
E03	Continuité des données	Données collier non accessibles au vétérinaire	7	4	4	112	Critique
T01	API Backend	Indisponibilité du backend (downtime)	8	4	3	96	Élevé
T02	Time-series DB	Corruption ou perte de données capteurs	8	3	4	96	Élevé
I04	Chat RAG vétérinaire	Indisponibilité ou latence excessive du LLM	6	4	3	72	Élevé

Seuils : IPR < 30 faible · 30–59 modéré · 60–99 élevé · ≥ 100 critique. La détection est inversée (1 = très détectable, 10 = indétectable).

15.2 Risk Map projet

La Risk Map positionne chaque risque projet selon son score **Score = P × I** (Probabilité × Impact, de 1 à 5). Registre trié par score décroissant.

ID	Catégorie	Risque	P	I	Score	Niveau
R01	Technique	Capteur FC non fiable sur l'animal	4	5	20	Critique
R02	Technique	Bus factor : Cyril seul sur le hardware IoT (firmware)	4	5	20	Critique
R07	Légal	LLM génère un conseil médical erroné : responsabilité juridique	3	5	15	Critique
R03	Technique	Données d'entraînement insuffisantes : modèle IA peu fiable	3	4	12	Élevé
R04	Technique	GPS : latence / dérive, temps réel non garanti	3	4	12	Élevé

ID	Catégorie	Risque	P	I	Score	Niveau
R09	Marché	Vétérinaire partenaire unique : rupture du partenariat	3	4	12	Élevé
R12	Humain	Départ ou indisponibilité d'un membre clé	3	4	12	Élevé
R13	Planning	Sous-estimation de la charge firmware : dérapage calendaire	4	3	12	Élevé
R05	Technique	BLE : perte de connexion collier-mobile	3	3	9	Élevé
R14	Planning	Retard du prototype collier (composants / impression 3D)	3	3	9	Élevé
R06	Technique	Dépassement du budget API LLM	4	2	8	Modéré
R08	Légal	Non-conformité RGPD (données de santé animale)	2	4	8	Modéré
R10	Marché	Faible adoption : propriétaires pas prêts à payer	2	4	8	Modéré
R11	Marché	Concurrence déjà positionnée (Tractive, Weenect)	4	2	8	Modéré

Zones : Score ≥ 15 critique · 9–14 élevé · 5–8 modéré · 1–4 faible.

POLITIQUE DE TRAITEMENT

Un plan de mitigation est rédigé pour chaque risque dont le score est supérieur ou égal à 9, rattaché aux actions de l'AMDEC et aux jalons de dé-risquage du planning. Les risques sous ce seuil sont surveillés sans plan dédié.

16 Annexes

16.1 Backlog complet · 66 user stories

L'intégralité des user stories, groupées par epic. Chaque story porte, dans l'outil de gestion (Jira), des critères d'acceptation au format Given/When/Then (exemple en partie WBS & fonctions). Périmètre et persona principal indiqués par epic.

Réf.	User story
Epic 1 · Onboarding, comptes & appairage 7 US · MVP · Propriétaire	
1.1	Créer un compte sécurisé (email + mot de passe)
1.2	Se connecter avec une session sécurisée
1.3	Réinitialiser son mot de passe
1.4	Créer le profil de son animal (nom, race, âge, poids)
1.5	Appairer le collier par QR code (BLE)
1.6	Gérer plusieurs animaux sur un même compte
1.7	Modifier ou supprimer un profil animal
Epic 2 · Collecte, transmission & stockage 6 US · MVP · Système / collier	
2.1	Collecter les mesures des capteurs à intervalle défini
2.2	Normaliser et dédupliquer les données reçues
2.3	Mettre les données en tampon local hors réseau
2.4	Retransmettre les données à la reconnexion
2.5	Stocker l'historique en séries temporelles
2.6	Authentifier le collier à chaque transmission (mTLS)
Epic 3 · Suivi bien-être & activité 4 US · MVP · Propriétaire	
3.1	Voir un score de bien-être quotidien
3.2	Consulter les courbes d'activité et de sommeil
3.3	Suivre les constantes (température, rythme)
3.4	Parcourir l'historique de santé
Epic 4 · Localisation & zones de sécurité 5 US · MVP · Propriétaire	
4.1	Localiser son animal en temps réel sur une carte
4.2	Définir des zones de sécurité (geofencing)
4.3	Recevoir une alerte de sortie de zone
4.4	Consulter l'historique des trajets
4.5	Activer le suivi temps réel (mode live)
Epic 5 · Détection d'anomalies & alertes santé 5 US · MVP · Système / propriétaire	
5.1	Établir un profil de comportement normal par animal
5.2	Ajuster des seuils d'alerte adaptatifs
5.3	Détecter une anomalie de comportement ou de santé
5.4	Recevoir une alerte santé compréhensible
5.5	Consulter le détail et l'historique des alertes
Epic 6 · Assistant IA & escalade 4 US · MVP · Propriétaire	
6.1	Poser une question à l'assistant IA (chat)
6.2	Obtenir une réponse fondée sur le corpus vétérinaire
6.3	Comprendre une alerte contextualisée par l'IA
6.4	Être orienté ou escaladé vers un vétérinaire si besoin
Epic 7 · Vet Portal · dossier & rapport 7 US · MVP · Vétérinaire	
7.1	Accéder à la vue patient d'un animal
7.2	Consulter l'historique médical chronologique
7.3	Lire un résumé contextualisé et les données brutes
7.4	Consulter les alertes de l'animal

Réf.	User story
7.5	Ajouter des notes au dossier
7.6	Exporter un rapport normalisé (PDF / JSON)
7.7	Suivre l'animal dans le temps (longitudinal)
Epic 8 · Abonnement, facturation & support 7 US · post-MVP · Propriétaire	
8.1	Choisir et souscrire un abonnement (Stripe)
8.2	Gérer le renouvellement de l'abonnement
8.3	Mettre à jour son moyen de paiement
8.4	Consulter ses factures
8.5	Résilier son abonnement
8.6	Contacter le support (ticket)
8.7	Consulter la FAQ
Epic 9 · Conformité, sécurité & RGPD 5 US · MVP · Propriétaire / système	
9.1	Donner ou retirer son consentement sur les données
9.2	Exporter ses données personnelles
9.3	Demander la suppression de son compte et de ses données
9.4	Bénéficier de l'anonymisation des données sensibles
9.5	Tracer les accès et actions (journal d'audit)
Epic 10 · Téléconsultation & triage 7 US · post-MVP · Propriétaire / vétérinaire	
10.1	Générer un PDF normalisé pour le vétérinaire
10.2	Être mis en relation avec un vétérinaire de garde
10.3	Recevoir une orientation non-diagnostique (triage)
10.4	Prendre rendez-vous avec un vétérinaire
10.5	Transmettre le dossier au vétérinaire (handoff)
10.6	Être routé vers son vétérinaire traitant
10.7	Rémunérer les vétérinaires via le pool plafonné
Epic 11 · Plateforme, livraison & observabilité 9 US · MVP · Système / Ops	
11.1	Déployer les services via CI/CD (GitOps)
11.2	Conteneuriser et versionner chaque service
11.3	Provisionner l'infrastructure en IaC (Terraform)
11.4	Superviser la santé des services (monitoring)
11.5	Collecter métriques, logs et traces (OpenTelemetry)
11.6	Être alerté en cas d'incident
11.7	Gérer les secrets de façon chiffrée
11.8	Sauvegarder et restaurer les données
11.9	Assurer la montée en charge (scalabilité)

16.2 Glossaire

Terme	Définition
Care Engine	Moteur d'IA conversationnelle de Pawrise Care (pipeline LangGraph 6 nœuds).
RAG	Retrieval-Augmented Generation : le LLM répond à partir de documents récupérés (corpus vétérinaire).
mTLS	Authentification mutuelle TLS : client et serveur se prouvent mutuellement leur identité par certificat.
GitOps	Déploiement piloté par Git : l'état du cluster suit ce qui est décrit dans le dépôt (Argo CD).
IaC	Infrastructure as Code : l'infrastructure décrite en fichiers versionnés (Terraform).
IPR	Indice de priorité de risque (AMDEC) = Gravité × Occurrence × Détection.
LTV / CAC	Valeur vie client / coût d'acquisition client.

Terme	Définition
LTE-M	Réseau cellulaire basse consommation pour l'IoT, avec mobilité.
Store and forward	Stockage local des données hors réseau, renvoyées à la reconnexion.

16.3 Références des décisions d'architecture (ADR)

Les dix ADR du Care Engine sont détaillées dans la partie « IA · le Care Engine ». Les choix d'architecture système et cloud sont justifiés dans les parties correspondantes (besoin, alternatives écartées, décision, trade-off).

16.4 Sources & outils

- **Marché** : [FACCO / Kantar 2024-2025](#), [Grand View Research](#) (pet wearables).
- **Prix cloud** : [Hetzner Cloud](#) (ajustement tarifaire du 15 juin 2026), [Scaleway Kapsule](#), [OVHcloud Managed Kubernetes](#).
- **Prix modèles IA** : [Azure OpenAI](#), [Cohere](#), [Mistral](#) (tarifs 2026).
- **Cadre légal** : Code rural, art. L243-1, RGPD, AI Act, GPSR (UE 2023/988), Règlement Batteries (UE 2023/1542).
- **Outils projet** : [Jira](#), [Confluence](#), [GitHub](#), [Figma](#). **Documentation** : [Typst](#) (tableaux générés depuis la base de contenu du prototype).

16.5 Prototype en ligne

Une version interactive de l'architecture (cockpit jouable), des diagrammes et de l'ensemble des contenus est disponible sur pawrise-care.com. Ce dossier en est la version académique, imprimable et autoportante.